

One-Step Schnorr Threshold Identification

Foteinos Mergoupis-Anagnostou

GRNET, Athens, Greece, fmerg@grnet.gr

Abstract. Threshold cryptographic primitives have not been widely adopted in real-world distributed systems (i.e., beyond the closed committee model), presumably due to state-synchronization overhead and complex certification processes for the shareholders. These are both aspects of their over-reliance on infrastructure, a hidden assumption that is usually glossed over in their design. In this work, we propose **OSST**, a Schnorr-based real-time threshold identification protocol that achieves non-interactivity and non-reliance on public shares by means of direct proof interpolation. Given a Shamir (n, t) -shared secret x , the proposed scheme allows any $t^* \geq t$ (but no less) shareholders to prove over designated communication channels that their secret keys interpolate to x without revealing any information beyond that. Provers do not engage in distributed computations, sending their packets to the verifier asynchronously; conversely, verifiers need only know the combined public key $y \equiv g^x$, without need to pre-validate and register the individual member identities. The protocol is intended for use in permissionless or unmanaged meshes that lack both overlay networks and trust infrastructure or governance, a use case space that has been tacitly neglected as “niche” by the current mainstream. No auditable multi-key setup is required beyond distributing x according to Shamir’s secret sharing (or equivalent distributed key generation scheme) and correctly advertising its public counterpart; in particular, the protocol is intended to be secure against impersonation attacks without relying on the consistency of any advertised public shares. We provide evidence that this has good chances to hold true by giving a formal security proof in the random oracle model under the one-more discrete-logarithm (OMDL) hardness assumption.

1 Introduction

Distributing a signature scheme is usually achieved through a trusted combiner who acts as group proxy. Upon every session, the shareholders engage in the multi-party computation of a commitment, which they next use to generate ordinary signatures with their respective keys. After aggregating these packets, the proxy combines them into an ordinary signature and forwards it to the verifier, who proceeds to verification like in the non-distributed case. The idea stems from [34] and has been elaborated in the proof-of-knowledge context by [21], who apply it to distribute protocols as general as proving the preimage of a homomorphism (cf. [23]). By distributing the prover of the Schnorr protocol and having the verifier coincide with the combiner, we get a real-time threshold identification scheme.

This protocol does not require prevalidation of the shares by the verifier but is heavily interactive on the provers’ side. This is due to the multi-party computation of the commitment, usually some distributed key generation (DKG) scheme, which requires ad hoc tuning and continuous maintenance. This is namely a bandwidth greedy and memory intensive operation with inherently high message and space complexity (cf. [12], [11]) that relies on stable

network connections among the provers. Standardized techniques for reducing bandwidth usage or the number of rounds (e.g., a FROST-like proving phase, cf. [22]) require extended infrastructure support and incur governance overhead. While these may not be an issue for certain managed consortia, they make interactivity prohibitive in unmanaged meshes or if the provers must operate asynchronously or under volatile (or absent) intra-group network connections.

Non-interactivity can be trivially achieved if the public shares are advertised and trusted. In this case, the provers send ordinary proofs for their keys to the verifier, who validates them against the respective public shares and accepts only if the latter interpolate to the group public key (i.e., the number of involved provers is at least equal to the threshold parameter). This scheme is asynchronous and requires no infrastructure support on the provers' side; however, it requires from the verifier to have pre-validated and registered the correct public shares in order to reliably verify the individual proof packets. This evidently implies the need for an auditable setup phase attesting to the consistency of a fixed set of shares, otherwise the protocol becomes susceptible to rogue-key attacks. If reliance on authorities is not desired, auditability becomes equivalent to persistent and trusted bulletin boards for advertizing the transcript of the setup distribution process, allowing candidate verifiers to anytime validate the acclaimed public shares against the group public key. Beyond scalability issues, security implications and the infrastructural costs of governance, such solutions require specifications for interoperability and have to address continuous administrative burdens like proactive sharing, which would require to update the boards and repeat validations upon every epoch from scratch. Finally, this scheme is essentially an access control mechanism for onboarding named shareholders with rigid identities through individual authentication –a problem much narrower than identifying a fluid group of shareholders in the permissionless setting.

One could circumvent pre-validation by having the provers attach their current public keys to their respective proofs upon every session; in this case, the verifier accepts only if every proof verifies against the attached public share and the attached shares interpolate to the combined public key. Security reduces intuitively to the knowledge property of the ordinary Schnorr protocol. Every party “should know” the logarithm of the attached public share because the respective proof verifies; since the attached shares interpolate to the combined public key, their respective logarithms interpolate to its secret counterpart and the provers comprise thus a qualified coalition. This protocol is both non-interactive and non-reliant on the public shares (if at the cost of increased bandwidth consumption and inefficient verification) but the knowledge argument regarding security is not as strong as it might seem. We will later argue why a formal security proof in the lines of this reasoning is in fact intractable.

1.1 Contribution

The above discussion indicates a tension between non-interactivity on the provers' side and non-reliance on the public shares; eliminating infrastructure on one side requires its existence and maintenance on the other. In this work, we propose a Schnorr-based real-time threshold identification protocol that achieves both properties¹ by means of direct proof interpolation. The protocol is intended for use cases where both the administrative overhead of certifying the shares and the engineering investment for reducing per-session interactivity are cost-ineffective or prohibiting, including edge cases where even advertising

¹ Bilinear threshold protocols (namely BLS, cf. [10]) can achieve non-interactivity and share-freeness as well; however, they are incompatible with and less computationally efficient than the widely deployed DL-based infrastructure. The present work seeks a solution within the pure discrete-logarithm setting.

shares is impractical and asynchronicity or operability under volatile (or absent) intra-group connectivity is prioritized. No auditable key setup is required beyond sharing a secret key according to some Shamir-based scheme (no matter which) and correctly advertising its public counterpart; in particular, the protocol is intended to be secure against impersonation attacks in the share-free model. We provide evidence that this has good chances to hold true by giving a reasonably restricted security proof in the random oracle model under the OMDL hardness assumption.

1.2 Organization

In Section 2, we investigate the overlooked area of real-time threshold identification and delineate the potential use case space of the proposed scheme. In Section 3, we introduce the protocol, examine its properties and make some security considerations, including a key-recovery attack against its interactive variant. In Section 4, we specify the threat model and derive the relevant attack games. We do not formulate zero-knowledge notions for the threshold setting, focusing directly on security against impersonation attacks; however, we derive a restricted notion of extractability (in the sense of special soundness) in order to specify a reasonably narrow attack model that makes a security proof possible under the OMDL hardness assumption. In Section 5, we employ the above mentioned key-recovery attack to provide a formal security proof in the random oracle model. In Section 6, we demonstrate the practical efficiency of the proposed protocol by means of a back-of-the-envelope calculation.

2 Survey on real-time threshold identification

Let $\mathbb{G} = \langle g \rangle$ be a cyclic group of order $q > 1$ where the discrete-logarithm (DL) hardness assumption holds true. Let a secret key $x \in \mathbb{Z}_q$ be distributed as $x_1, \dots, x_n$ among a group of shareholders by means of Shamir’s (n, t) -sharing, $1 \leq t \leq n$, or equivalent DKG scheme (cf. Section 2.1.1). That is, it shouldn’t matter if x has been shared by a dealer or is implicitly defined in a decentralized fashion; it suffices that the output of the distribution scheme is computationally indistinguishable from that of Shamir’s secret sharing. We are interested in a protocol that allows any $t^* \geq t$ (but no less) shareholders to dynamically prove that they belong to the group entity represented by $y \equiv g^x$. More accurately, the involved parties should be able to collectively prove in a real-time session bound to designated communication channels, which have been separately opened between the verifier and the individual provers, that their secret shares combine to x in the sense of interpolation (cf. Remark B.3).

Security against impersonation attacks The constraint of separate and designated communication channels is essential. No matter if they need to collaborate, the provers have to independently send their final proof packets through the specific channels that they have separately opened to the verifier upon, say, some initial identification request; this ensures that the verifier identifies the actual group members that appear lively over the network and not a delegated group proxy that could instead present a combined threshold signature bound to a session nonce (cf. Section 2.4). Security against impersonation attacks means that any probabilistic polynomial-time adversary controlling up to $t - 1$ shareholders remains unable to execute the protocol and cause the verifier to accept with non-negligible advantage.

As a hypothetical application, consider a distributed SSH session where at least t -out-of- n members of a network operations team must be simultaneously connected to be able to

operate. Following some threshold signature scheme, a qualified coalition of them sign the session identifier with their secret keys using a common commitment (cf. Section 2.3.1) and send their packets to the server directly, who combines them into an ordinary signature and verifies it against the team authorized key. This allows the server to authenticate the ensemble in real-time over the currently established channels, evidently extending the SSH protocol. However, this natural extension requires the shareholders to be state-synchronized and interact in real-time in order to generate a distributed commitment that is bound to the current session; before even considering latency and robustness, this is prohibitive just because no machine-to-machine connections can be directly established between them (unless the SSH framework provided native peer-to-peer capabilities). As a non-interactive alternative, suppose that the public shares of the individual members are installed on the server along with the team authorized key as a fixed sharing of it; in this case, the server can individually verify the independently generated signatures from the involved shareholders, contending itself with the fact that their public keys interpolate to the public key of the team. However, this contains non-trivial assumptions that incur even more complications in the already risky landscape of SSH key management; specifically, either the SSH daemon must validate by cryptographic means that the provided shares constitute indeed a (n, t) -sharing of the authorized key, or the server administrator trusts them because it trusts the providers. In either case, updating the installed shares introduces operational burdens; even revoking a single share is a system-wide event that necessitates administrative action. The threshold condition becomes artificial as it requires monitoring, alertness and governance in order to become securely effective in the long term.

A similar problematic can be traced in almost every imaginable use case of real-time thresholds. Observe that threshold SSH belongs still to the closed organization model, where managing the lifecycle of public shares can sometimes be tolerated and even preferable. This is not so in open settings, where verifiers are not subject to the same management as the shareholders. Consider an online allocated critical resource, e.g., a private group chatroom or quotas in a high-performance computing server, that should not become available to a registered group until t -out-of- n members request it. Or, more generally, any network where certain nodes should not release sensitive resources or actions unless t -out-of- n shareholders give their signal for it. Relying on trusted shares presupposes that these nodes have reference to some authoritative source attesting to their consistency as a fixed sharing of the group public key. Even if a web of trust is employed in place of authorities, the administrative burden in case of resharing and revocation remains prohibitive, while the pure cryptographic means by which their consistency becomes certified requires overwhelming infrastructure investment and maintenance. Conversely, if the shares are not advertised and trusted, practicality can be severely limited if the provers need to interact and state-synchronize upon every session. The simplest reason is the coordination overhead, which becomes unbearable unless some low-latency overlay network is maintained through continuous engineering effort. Apart from network infrastructure costs, requiring from provers to be state-synchronized is a constraint that may be irrelevant or antithetical to the use case; joining a session or giving consent to an action should not require to participate in session initiation and bind to the same commitment as everybody.

There exist edge but imaginable use cases where requiring from provers to interact with each other is not only restrictive or inconvenient but physically prohibitive –even if machine-to-machine connections are in principle possible between them. Consider a volatile mesh of globally distributed sensors, responsible for transmitting signals to a gateway-node under ultra low power and adverse weather conditions; or a vehicular fleet that needs to

notify some command-and-control unit within opportunistic connectivity windows. In such scenarios, provers lack the luxury of intra-group real-time communication due to intermittent or absent mutual connections; they cannot operate unless able to generate offline their proof packets and send them whenever possible to the verifier asynchronously. This is one aspect of generalized non-availability, the primary causes of which –temporary device failure, partial network collapse and radio signal obstruction or degradation– can equally affect the trusted share infrastructure. A space unit that trusts outdated shares may be unreachable for receiving updates within the critical window that it needs to identify a system of satellites; just like a gateway-node that recovers the combined public key of a wireless sensor network after some natural disaster event may be unable to reach the bulletin board that advertizes the public shares and attests their consistency.

Below we revisit known primitives and establish a conceptual framework for analyzing the tradeoffs of real-time Schnorr thresholds, arguing that native and system-agnostic schemes should be share-free and non-interactive. As already hinted, the necessity of share-freeness follows from the general impracticality of trusted share governance and is essentially akin to the permissionless setting, i.e., the natural setting of identification. That same setting implies that a particular denial-of-service attack is endemic to share-free real-time thresholds, even if these are interactive (e.g., adapted Schnorr threshold signatures); in fact, non-interactive thresholds have better chances to be eventually resilient against this type of attack exactly due to their asynchronous nature. Finally, we observe that both interactivity and share-reliance can be overcome through the direct interpolation of proofs that can be independently generated and asynchronously sent to the verifier.

2.1 Group constitution

The most elementary way to create a (n, t) -threshold group, $1 \leq t \leq n$, is when a single dealer distributes a pre-existing key by means of Shamir’s secret sharing (cf. Appendix B), i.e., a centralized operation of the form

$$(x_1, \dots, x_n; y) \leftarrow D(g, x). \quad (2.1)$$

This corresponds to a scenario where the key holder splits x across many devices for the purpose of distributed custody, or in order to secure it against loss or a single point of compromise; more generally, it indicates a redundancy or decentralized access mechanism, with the level of trust upon the dealer depending on its exact role within an organization. In case of limited trust, the shareholders need to verify that their received shares are consistent, i.e., that they comprise indeed a (n, t) -sharing of $x = \log y$. This is attained by extending raw secret sharing into a *verifiable* secret sharing (VSS) scheme ([13], [27]), i.e., by having the dealer commit to the interpolating polynomial and broadcast its commitments, incurring $O(n)$ inbound bandwidth and $O(nt)$ local verification costs per shareholder.

2.1.1 Distributed key generation (DKG)

In several cases, the combined secret key must be implicitly defined in a decentralized fashion; i.e., it should not be known to any single dealer (or be reconstructible at any single location), with the shareholders contributing to it equally. This is attained by leveraging the homomorphic property of Shamir’s secret sharing (cf. Remark B.6). Every group participant splits some uniformly random secret across its peers –including itself– and sums the shares received from the others, with the local summation process yielding its final secret share; after

all broadcasts have been completed, the peers are able to infer the same group public key y , but x remains completely unknown to everybody. In case of zero trust, these sharings must be VSS-backed, so that everyone can verify the correctness of their received packets. The resulting scheme is called a (n, t) -distributed key generation (DKG) scheme; it is equivalent to Shamir’s secret sharing in the sense that the probability distribution of its output shares is computationally indistinguishable from the output distribution of (2.1) with $x \leftarrow_{\$} \mathbb{Z}_q$.

Unoptimized DKGs ([14], [31]) have high message and space complexity ($O(n^2)$ per node) since every party must broadcast its own polynomial commitments while downloading those of its peers; this incurs measurable latency and is prone to packet loss or disruptions, making them practical only for small n ’s. However, recent optimizations suggest adoption paths for moderate to large systems (cf. [19], [12], [11], [17], [35]; an overview can be found in [2]). Such improvements include reducing bandwidth usage or mutual verification costs by order of magnitude, or reducing the original message complexity from quadratic to almost linear, but come at the cost of sophisticated engineering and ad hoc tuning (hierarchical or topological designs, asynchronous overlay networks, parallel connections and batching, etc.). In this regard, [12] introduces a formal engineering framework that streamlines the optimization process, with its subsequent improvement [11] reducing total bandwidth consumption by 80% (under the same cubic complexity); however, the engineering effort for maintaining a secure and low-latency consensus network remains substantial (including dynamic participation management to ensure that a two-thirds majority of honest nodes remain always online). This must be particularly taken into consideration if DKG’s have to be repeatedly executed to generate a combined commitment per session (cf. Section 2.3).

2.2 Reliance on public shares

A distributed protocol is said to be secure in the *plain public-key* model (cf. [5]) if the public shares $y_i \equiv g^{x_i}$, $1 \leq i \leq n$, need not be certified in any sense; specifically, the shareholders need not undergo validation processes for their secret keys (i.e., proofs of possession or knowledge) because the protocol is immune to rogue-key attacks by design. In the extreme, the public shares may not even be advertised to the outer world; in this case, the only static input to verification is the public key y , a property that will be referred to as *share-freeness*.

2.2.1 Universal share certification

Strong reliance on the public shares means that the verifiers have to trust and register them; consequently, in a threshold system, the certification process must generally attest their consistency, i.e., that they comprise indeed a sharing of the group public key. How this is achieved depends on the use case and the exact sharing protocol. In a supposedly closed committee, where public auditability does not matter or the role of verifier is reserved for specific shareholders, it suffices that the public shares be advertised and trusted within the group; certification amounts to the setup phase DKG (cf. Section 2.1.1), which allows the peers to one-off validate each other’s key on the fly, and the relevant infrastructure confines to the local registries where each node maintains the public keys of its peers. In more transparent settings, where verifiers and monitors reside outside the group, the distribution process has to be publicly auditable in the long run, which comes at the cost of extended trust infrastructure. Even under centralised sharing, where the shares are essentially controlled by a dealer, the latter must employ some *publicly* verifiable secret sharing (PVSS) scheme ([33], [31]) and have its transcript persistently stored in a bulletin board in order to make it accessible to any future verifier or monitor; specifically, external entities must be able

to download ($O(n)$) and verify ($O(n^2)$) this transcript in order to trust and register the advertised public shares.² The administrative burden remains the same in the canonical peer-to-peer case, where instead of centralised sharing a PVSS-backed DKG has to occur; in this case, the transcript consists of the transcripts produced by the totality of shareholders, which yields increased space complexity ($O(n^2)$) and a primary scalability bottleneck.³

Share certification has not found its way to industry⁴ even for the case of small n 's. The reason lies beyond transcript size, asymptotic verification costs and the engineering investment required from auditors. Any effort to standardize a “public-share infrastructure” would need to modify the Web PKI standards (e.g., by specifying a group-centric certificate signing request model and extending the existing certificate formats) or the web of trust conventions. Even if backwards compatibility were not required, it would still need to rely on persistent and trusted bulletin boards, thereby increasing its governance costs and attack surface, and further address the administrative burden of proactive sharing, which would require to update the boards and repeat validations upon every epoch from scratch.

2.2.2 Share-freeness

We have remarked that share-freeness is a rather radical way to achieve security in the plain public-key model. Indeed, the verifier of a distributed protocol can strongly rely on advertised shares and yet these may need not be certified; the typical example is the Bellare-Neven multi-signature scheme [7], where security against rogue-key attacks is achieved without requiring from the individual parties to prove knowledge of their secret keys during the key setup phase. However, if some threshold condition is to be further satisfied (i.e., the protocol is based on Shamir's sharing), security against rogue-key attacks must include the inability of an attacker to imitate a false sharing of the group public key. Since fabricating consistent public shares for any given public key is basically trivial, the verifier must necessarily consult a trusted bulletin board attesting to the consistency of some fixed sharing before registering the individual identities for a given group of shareholders. That is, for threshold protocols, reliance on public shares cannot circumvent certification in the sense of Section 2.2.1; to the extent to which this remains impractical beyond the closed committee model, share-freeness emerges as the one plausible alternative for system-agnostic threshold identification protocols.

2.3 Interactivity

The recurring network overhead of per-session interactivity is often the critical bottleneck in threshold protocols, hindering them from scaling to large n 's and arbitrary t 's. Note that simply establishing secure channels between peers can itself be costly and fragile, while the latency from multi-round hops among globally distributed nodes can accumulate rapidly before the cost of cryptographic operations is even considered.

² Recent work in [3] reduces the persistent storage requirement by nearly 40% and the asymptotic costs of verification to $O(n)$ without essential overhead.

³ Lightweight verification techniques can theoretically mitigate this but introduce significant and context-dependent tradeoffs. For example, KZG commitments (cf. [20]) reduce transcript size to constant but require a trusted setup within the bilinear setting; similarly, SNARKs (cf. [16]) achieve constant time but impose prover overhead along with engineering investment for circuit design. Optimistic verification achieves constant costs but sacrifices universality by delegating trust to a designated verifier cohort, further necessitating ad hoc engineering along with incentive and slashing mechanisms.

⁴ No existing or emerging applications seem to be reported except for a few ledger-related proposals and digital voting prototypes. These are mostly irrelevant to a standardization effort; moreover, this topic is not explicitly addressed in the NIST 2025 MPTC Project [25].

2.3.1 Compactness

Interactivity of Schnorr-based protocols arises from the need to generate a combined commitment to be used by the provers. This serves a double purpose; it ensures that the threshold condition is satisfied by having all provers bind to it and it allows their packets to be compactly aggregated into an ordinary signature that can be efficiently stored and transferred. These mechanics are outlined in Section 2.10.1.

The commitment is generated as the public output of a DKG (cf. Section 2.1.1), which is a bandwidth greedy and memory intensive operation that does not scale without tuning. While a system of nodes may be able to sustain one-off or occasional DKG's (e.g., for the purpose of generating their combined key or refreshing their shares during stable connectivity windows), continuously depending upon it might be infeasible; the simplest reason is the engineering effort required to maintain a secure and low-latency overlay network. Moreover, despite attaining moderate to high scalability, practical DKG's still face barriers that currently hinder them from becoming network and device agnostic solutions; for example, the state-of-the-art [11] reports 3.32 MB of outbound traffic per node ($O(n^2)$) and 12.48 seconds of average execution time already at $n = 64$, where nodes are globally distributed lightweight servers operating under realistic but relatively stable network conditions. These benchmarks indicate massive delays and packet loss in a volatile mesh of memory-constrained devices.

In practice, the commitment is generated as in the signing phase of the FROST protocol ([22]), which achieves linear bandwidth per node and reduces the number of rounds to one or two, depending on whether a semi-trusted aggregator is employed or not. Production-grade simulations indicate that the protocol remains efficient for several hundreds of nodes.⁵ This is attained by means of a preprocessing stage, where every node generates nonce pairs and registers the respective commitments in the aggregator's datastore, allowing subsequent proving phases to begin without preparatory rounds. This imposes an upfront operational overhead of synchronization and potentially a continuous storage burden per node, since peers have to securely retain their precomputed nonces; moreover, system availability depends crucially on this inventory, with the proving functionality being suspended in case of nonce depletion until the next epoch. While standardized, widely implemented and the closest possible to non-interactive, FROST requires stable intra-group connectivity and faces challenges in volatile or unmanaged settings, where the scheduling of precomputations and periodically synchronizing the aggregator may be impossible or cost-ineffective.

This discussion implies that generating combined commitments can be a difficult task that requires context-specific recipes. However, the compactness constraint may not be crucial in most identification settings, where the benefits of non-interactivity can outweigh those of producing constant-size audit trails; in this case, the primary justification for interactivity reduces to the threshold constraint, and we are thus motivated to ask if this can be achieved in non-trivial ways without intra-group interaction. Indeed, non-interactive protocols are more likely to be robust; in the extreme, they remain applicable when provers are compelled to operate under volatile or absent intra-group connectivity.

2.3.2 Asynchronicity – Incremental verification

After group constitution, the shareholders may be unable to discover each other, e.g., because no traceable channels persist among them or intra-group connectivity is intermittent due to

⁵ [15] reports practical efficiency at $n = 1000$ and $t = 667$, with nodes running on Linux-capable devices with consumer-grade high-performant processors.

uncontrolled factors. Even if communication remains possible, an application may require the provers to operate without being simultaneously online for the purpose of interaction.

Non-reliance on real-time interaction means that proving sessions are non-interactive on the provers' side, imposing a high degree of asynchronicity on their interaction with the verifier (unless the latter is able to interactively coordinate them); specifically, the verifier's interaction with the j -th shareholder should not depend on any packet received from the i -th shareholder, $i \neq j$. Note that this scheme facilitates *incremental verification*, i.e., the capability of a prover to attach itself to an open session without having the already identified coalition to rerun the proving phase and be state-synchronized with them.

2.4 Real-time verification

Despite being routinely employed as real-time identification protocols, signature schemes per se do not require from the signer to communicate with the verifier in real-time; the verifier needs only ensure that the acclaimed holder of a secret key has undeniably approved a specific message, no matter if the relevant proof (i.e., the signature) is sent by the signer through an open channel, carried by designated entities or persistently stored in a ledger. Only when the verifier interacts with the signer in real-time, asking it to sign a nonce of its choice, can a signature scheme serve the purpose of identification over a designated communication channel. Note that real-time interaction may not be direct, i.e., the signature may be forwarded to the verifier by a sequence of proxies, which are in turn responsible for relaying back the response of the verifier. For non-distributed protocols, trusting these proxies amounts to trusting them as network relay nodes, i.e., that they do not obstruct communication intentionally and they cannot tamper with the transmitted content (e.g., because SSL/TLS is enforced between network hops).

2.4.1 Untrusted group proxies

The above remarks can be generalized with some nuance to the distributed setting. First, the objective of threshold identification is to establish trust against a live coalition of unknown entities that appear across designated communication channels; a threshold signing primitive can be employed in this context only if the verifier asks from the provers to sign a nonce of its choice for the sake of synchronizing states. Second, threshold signing primitives usually involve a combiner, who acts as group proxy by aggregating the individual packets into a compact signature and forwarding this to the verifier. However, this gateway is not a generic relay node, it runs ad hoc cryptographic software that is deployed by trusted intermediaries or internally maintained by dedicated group staff; in either case, trusting this service is a highly non-trivial assumption. In order to remove this assumption, the verifier must interact with the individual provers directly, i.e., at most through generic relay nodes; specifically, whenever employing [34] or FROST [22] for identification, the role of combiner has to be reserved for the verifier (cf. Section 2.10.1). Given this constraint, the particular performance benefit that arises in threshold signing primitives by separating the combiner from the verifier, i.e., the constant-time verification of a constant-size signature, is irrelevant for real-time thresholds; the verifier has to aggregate and process $O(k)$ distinct packets anyway ($k = |Q|$).

2.4.2 Direct interaction

More generally, in order for real-time threshold identification over designated communication channels to make sense, the provers must send their cryptographic packets over those same

channels that have been directly opened to the verifier (i.e., at most through generic relay nodes). Even if their packets are the aggregable product of some intra-group multi-party computation, no special gateway node can generally be assumed to forward their aggregation to the outer world; this would at best identify that node as a delegated proxy acting on behalf of a qualified coalition, not the live coalition of shareholders itself.

2.5 The permissionless setting

Racall that the verifier must interact with the provers in real-time without intermediaries (cf. Section 2.4.2) and without generally knowing their individual identities (cf. Section 2.2.2). Even if the provers present acclaimed public keys, the verifier can at best validate them on the fly (e.g., by requiring individual proofs of knowledge); if these keys further comprise the actually fixed sharing of the group public key, the verifier has generally no way to validate this fact because universal share certification is not an option (cf. Section 2.2.1). More generally, beyond the closed committee model, an authority cannot correlate any acclaimed identity with an actual shareholder unless it has taken part in the group setup as a peer (or dealer) and is thereby able to “know” in some sense that shareholder. Consequently, for open decentralized systems that keep reliance on authorities to a limited scope, it makes sense to assume that the provers appear completely unvetted, i.e., that the verifier has zero prior knowledge on the entities it interacts with. Contrary to threshold signing primitives, where the shareholders can operate in a permissioned environment to output a static signature, the natural setting of threshold identification is a permissionless network where entities participate without any form of accreditation and are free to open unsolicited connections to the verifier as self-proclaimed shareholders.

2.6 Session poisoning

The permissionless model implies that share-free threshold protocols are inherently susceptible to a stealthy denial-of-service attack at the application layer, obstructing a qualified coalition $Q \subset \{1, \dots, n\}$, $|Q| \geq t$, of legitimate shareholders from having their ensemble identified. An attacker posing as the j -th shareholder submits a structurally valid but semantically incorrect proof packet to the verifier. In the absence of universal share certification mechanisms (cf. Section 2.2.1), the verifier has no authoritative reference to the public share y_j ; in particular, it has no reliable way to detect the malicious packet by pre-validating it against y_j and thus exclude it from verification. The coalition $Q^* = Q \cup \{j\}$ fails to identify and consequently service is denied to Q .

Interactive fault isolation Assuming that intra-group communication is state-synchronized with the verifier (so that the attacker cannot make unsolicited submissions without having talked to the other provers) and the public shares are known among the shareholders for mutual verification purposes, then a honest majority of peers can detect the malicious actor during the proving round and notify the verifier to separately deny service to j without disrupting the session for the other provers. However, this requires a robust consensus and notification mechanism while also relying on subtle security assumptions regarding the majority of honest peers; indeed, maintaining a secure consensus network requires active participation management and has itself a notoriously broad attack surface. Finally, such solutions would fail to detect an actual but malevolent shareholder and are obviously non-applicable in non-interactive contexts.

Reactive fault detection Asynchronicity facilitates a potentially better way to separately deny service to j by delegating detection duties to the verifier. Suppose that the verifier has identified some baseline coalition Q and applies incremental verification (cf. Section 2.3.2); by asynchronously onboarding j to the current session, the verifier can reliably infer that this packet causes the verification condition to fail for Q^* and thus drop the respective connection. This countermeasure makes non-interactive protocols immune to session poisoning after a baseline coalition has been identified, with its feasibility depending on how fast the verifier can detect such a coalition within a pool of aggregated packets. Obviously, bulk verification should here be as efficient as possible since it may need to be repeated multiple times until a baseline coalition is detected. This is basically trivial if the malicious packets are few, in which case the protocol is practically immune from the outset; however, in case of flooding, uncritically exhausting combinations becomes itself an attack vector because it drains the server resources and precipitates a full denial-of-service. To avoid this, reactive fault detection should employ backtracking algorithms that reduce the possibilities of combinatorial explosion and be combined with rate limiting, CPU throttling or other system-level techniques that ensure abortion if some resource threshold is critically exceeded.

2.7 Accountability

The above DoS attack is the problem of non-accountability within the permissionless setting. Accountability measures the capability of a system to detect which specific participants behave maliciously during a protocol round. This matters in use cases where responsibility must be attributable to named actors (e.g., financial consortia or hierarchical committees) or details on malicious activity must be gathered for the purpose of forensic investigations. Malicious actors can be shareholders who deviate from the protocol or intruders who try to impersonate legitimate shareholders; in either case, from the cryptographic viewpoint, they should be identified by reference to the acclaimed public key which an interested party uses to validate their activity. This evidently requires that the public shares be to a certain degree known and certified.

2.7.1 Intra-group vs extra-group

The auditability scope of certification relates directly to the desired accountability model. In the intra-group model of early rejects, i.e., if the protocol is interactive and the provers are responsible for detecting cheating peers during the proving session, it suffices that the public shares be advertised and trusted within the group; for threshold systems, this amounts to the setup phase DKG (cf. Section 2.1.1), which allows the shareholders to one-off validate each other's key on the fly, and the relevant infrastructure confines to the local registries where each node maintains the active public keys of its peers. In the extra-group model, i.e., if accurate blaming has to be accommodated on behalf of the verifier, the public shares have to be publicly auditable in the long run; for threshold systems in non-closed settings, this amounts to universal share certification in the sense of Section 2.2.1. The recurring network overhead of early rejects is replaced by the infrastructural overhead of trusted bulletin boards.

Extra-group accountability can be feasible for closed systems that need not interoperate with external auditors or tolerate authorities. In the first case, PVSS bulletin boards become practical (provided that n does not grow excessively and the group of shareholders remains relatively static) because the system does not need to comply with external standards. In the second case, the public shares are trusted because they are advertised by a designated group proxy that is widely recognized as such and entitled to perform this action. Systems

that prioritize decentralization will naturally opt for PVSS-based solutions; however, for open systems, such solutions amount to universal share certification, which we have seen to be infeasible for an array of compelling reasons (cf. Section 2.2.1). In short, extra-group accountability is unattainable in open decentralized settings due to the general impracticality of universal share certification. At the same time, these are the environments where extra-group accountability is mostly irrelevant par excellence; indeed, the underlying permissionless setting (cf. Section 2.5) implies that externally attributing responsibility to named actors by reference to their acclaimed public key is pointless, since these are generally free to acclaim any identity without vetting.

For open decentralized systems, intra-group accountability remains as the only practical approach to accountability and thereby an argument in favour of interactive protocols. Nevertheless, the operational benefits of asynchronicity outweigh by far those of early rejects. In fact, the sole justification for early rejects is the inadequate mitigation of session poisoning through interactive fault isolation, a stateful and fragile consensus mechanism with broad attack surface and strong infrastructural prerequisites.

2.8 Efficiency considerations

The wireless transmission of a single bit requires more power than a 64-bit (or 32-bit) CPU instruction by orders of magnitude (cf. [4]); that is, for low-power provers performing few exponentiations per round, packet size can be a critical factor regarding energy consumption on their side. Nevertheless, the number of transmitted packets dominates due to the high overhead of the radio transition states, implying that message complexity has probably greater impact than bandwidth consumption. For constrained meshes, this speaks strongly in favour of asynchronous protocols that avoid intra-group interactivity altogether.

A verifier that performs multiple exponentiations per session faces a computational energy cost that probably dominates any reduction in the number or size of incoming packets. However, reducing inbound traffic can be crucial in high-throughput scenarios where a server-class verifier handles multiple sessions concurrently. Under high load, even a marginal waste in bandwidth could precipitate network interface saturation; any per-session reduction can mitigate this risk by directly translating to an increase in throughput, provided that the processor of the network interface remains unaffected. Similarly, saving bandwidth is important for minimizing overall network congestion (since packets may need to be massively transmitted through relay nodes in order to be verified at arbitrary locations) by mitigating packet loss and consequently reducing the number of retransmissions.

2.9 Target use cases

Real-world systems can exist in a spectrum of trust and interactivity infrastructure, where pure solutions represent absolute tradeoffs and optimal recipes are hybrid relaxations or combinations. Interactivity (cf. Section 2.3) favours compactly aggregable proof packets and intra-group accountability by means of early rejects, while share-reliance (cf. Section 2.2) facilitates accurate blaming on behalf of external verifiers and monitors. In this regard, we seek a primitive that eliminates interactivity and reliance on trusted shares altogether, if to the detriment of accountability and compactness. This tradeoff accommodates use cases where both the administrative overhead of certifying shares and the engineering effort for reducing per-session communication are cost-ineffective or prohibiting, including the edge cases where operability under volatile or absent intra-group connectivity is prioritized. While this combination is seemingly an extreme constraint, it turns out to be the ideal

baseline of open decentralized environments and thereby a problem to be addressed by any system-agnostic threshold identification protocol. Indeed, we have seen that share-freeness (cf. Section 2.2.2) is the one plausible alternative as far as universal certification remains generally infeasible and extended reliance on authorities is not desired, while the benefits of asynchronicity (cf. Section 2.3.2) –greater robustness, lower latency, resilience under volatile connectivity, independence of action and incremental verification– can obviously outweigh the need for compact audit trails and interactive fault isolation (essentially an expensive and inadequate defense against the endemic threat of poisoning). These remarks are even more relevant for constrained or unmanaged meshes, where tuning and governance can be a limited or no option; even if ad hoc measures can be applied to a certain scope according to the specific use case, an agile identification scheme should be easily deployable under adverse or emergent conditions. In this regard, share-free and non-interactive thresholds emerge as the least common denominator of real-time operability and a resilient fallback mechanism for the case of systemic failure.

2.10 Existing approaches and tradeoffs

In this section, we examine more closely the existing paradigms. We denote $k = |Q|$.

2.10.1 Combined commitment approach

The Schnorr signature is a proof-of-knowledge for the signing key with a message baked into the hash function. This suggests a threshold identification protocol in the lines of [34]. The coalition of shareholders $Q \subset \{1, \dots, n\}$, $|Q| \geq t$, execute some DKG-like process to distributively generate a (n, t) -reconstructible commitment

$$u \equiv g^r, \quad r = \sum_{i \in Q} \lambda_i r_i,$$

where $\{\lambda_i\}_{i \in Q}$ are the interpolation coefficients and the nonce shares $\{r_i\}_{i \in Q}$ are held in private. Next, they generate proofs for their respective keys with a common challenge, i.e.,

$$(u_i, s_i), \quad s_i \leftarrow r_i + c x_i, \quad u_i \leftarrow g^{r_i}, \quad c \leftarrow H(u),$$

and send them to the verifier; after aggregating (u_i, s_i) , $i \in Q$, the verifier computes

$$s \leftarrow \sum_{i \in Q} \lambda_i s_i, \quad u \leftarrow \prod_{i \in Q} u_i^{\lambda_i}, \quad c \leftarrow H(u)$$

and accepts only if the aggregate proof (u, s) satisfies $g^s = y^c u$. Indeed, since $|Q| \geq t$, double application of Shamir's interpolation formula (cf. Remark B.3) yields

$$g^s = \prod_{i \in Q} g^{\lambda_i s_i} = \prod_{i \in Q} g^{c \lambda_i x_i} \prod_{i \in Q} g^{\lambda_i r_i} = \left(\prod_{i \in Q} y_i^{\lambda_i} \right)^c \prod_{i \in Q} u_i^{\lambda_i} = y^c u.$$

The protocol is share-free and interactive, producing a compact audit trail (u, s) . Verification requires $k + 2$ exponentiations and $O(k^2)$ field multiplications from the computation of

$\{\lambda_i\}_{i \in Q}$. Inbound traffic grows linearly with respect to k ; this is because the verifier must coincide with the combiner due to the constraint of real-time verification (cf. Section 2.4). Evidently, incremental verification is impossible because the session is bound to the combined commitment u that must be used by all provers. Efficiency depends on the exact scheme employed for generating u , i.e., some asynchronous DKG (cf. [12], [11]) or FROST-like process; an overview of their tradeoffs can be found in Section 2.3.1. FROST is particularly apt due to its linear scaling, provided that the infrastructural overhead of maintaining aggregators (along with the administrative burden of periodically synchronizing them) is not an issue.

The protocol facilitates interactive fault isolation against session poisoning. Specifically, if the provers know each other's public key, they broadcast their proofs to their peers for mutual verification before sending them to the verifier; in this case, a majority of honest peers agree on which packets are bogus and notify the verifier to reject the respective connections. We have stressed that maintaining a robust and secure consensus and notification mechanism for this purpose can be infeasible, inadequate and cost-ineffective, while the overhead from broadcasting or mutual verifications can obviously nullify the performance advantage of FROST over asynchronous DKGs. Delegating pre-validation to the combiner (as is the case with threshold signatures) is meaningless because the latter coincides with the verifier; the verifier would need to know and trust the individual public keys in advance, in which case real-time interaction among the provers becomes redundant (cf. Section 2.10.2).

2.10.2 Trivially decoupled approach

Non-interactivity can be trivially achieved if the public shares $y_i \equiv g^{x_i}$, $1 \leq i \leq n$, are presumed to be known and trusted by the verifier. In this case, the coalition of shareholders $Q \subset \{1, \dots, n\}$ can generate independent Schnorr proofs for their respective secrets, i.e.,

$$(u_i, s_i), \quad s_i \leftarrow r_i + c_i x_i, \quad c_i \leftarrow H(u_i), \quad u_i \leftarrow g^{r_i}, \quad r_i \leftarrow_{\$} \mathbb{Z}_q, \quad i \in Q$$

and send them to the verifier, who accepts only if the condition

$$|Q| \geq t \wedge g^{s_i} = y_i^{c_i} u_i, \forall i \in Q, \quad c_i \equiv H(u_i),$$

is satisfied. This scheme is asynchronous but share-reliant; in fact, its security relies entirely on the assumption that $y_1, \dots, y_n$ comprise the currently active (n, t) -sharing of y . Attesting to their consistency is a non-trivial task, whose cryptographic solution remains impractical in open decentralized settings for a variety of reasons (cf. Section 2.2.1). In fact, this artificial protocol is appropriate only for closed committees, where verifiers are group participants and have thereby validated the public shares during the key setup phase, or other closed managed environments that tolerate authorities and rely on certified updates. It is essentially an access control mechanism for onboarding the shareholders to a permissioned environment, allowing them to perform further threshold-cryptographic operations in an accountable fashion (e.g., signing). Security is dominated by the administrative burden of updating the verifiers' registries upon resharing; even revoking a single share requires system-wide administrative action.

2.11 Completely decoupled schemes

Assume that the public shares are not advertised and pre-validated; instead, they are attached to the proofs upon every session, allowing verifiers to validate them on the fly before checking

that they interpolate to y . That is, the shareholders $Q \subset \{1, \dots, n\}$ generate

$$(u_i, s_i, y_i), \quad y_i \leftarrow g^{x_i}, \quad s_i \leftarrow r_i + c_i x_i, \quad c_i \leftarrow H(u_i), \quad u_i \leftarrow g^{r_i}, \quad r_i \leftarrow_{\$} \mathbb{Z}_q$$

respectively and send them to the verifier, who accepts only if the condition

$$y = \prod_{i \in Q} y_i^{\lambda_i} \wedge g^{s_i} = y_i^{c_i} u_i, \quad \forall i \in Q$$

is satisfied. Security reduces intuitively to the knowledge property of the ordinary protocol. Every party “should know” the logarithm of the attached element because the respective proof verifies; since the attached elements interpolate to y , their logarithms interpolate to x and the provers comprise thus a qualified coalition. This protocol is share-free and non-interactive, albeit at the cost of less efficient verification.

The problem with this scheme is that it does not admit a formal security proof under (OM)DL hardness. More accurately, formalizing the “should know” argument by means of fork and extraction appears elusive. Note first that a proof of this style is here necessary; a straight-line reduction to the Schnorr protocol is not possible because the provers commit to unrelated group elements. Instead, some relevant extraction procedure should itself be reproduced within a forkable impersonation attack against the interactive variant of the protocol. Specifically, a black-box adversary $\mathcal{A}$ who succeeds in its impersonation attempt with non-negligible advantage while only knowing $t - 1$ secret shares should also spit the solution to the (OM)DL-problem with non-negligible advantage by means of forking. We proceed to show why this does not work.

The attack model for $\mathcal{A}$ is as follows. Initially, $\mathcal{A}$ chooses $J \subset \{1, \dots, n\}$ with $|J| = t - 1$ and sends it along with some $j \notin J$ to its challenger $\mathcal{C}$. Denote $Q = J \cup \{j\}$. $\mathcal{C}$ generates uniformly a Shamir (n, t) -sharing $x_1, \dots, x_n$ of some $x \leftarrow_{\$} \mathbb{Z}_q$ and forwards $y \equiv g^x$ along with $\{x_i\}_{i \in J}$ to $\mathcal{A}$. Subsequently, $\mathcal{A}$ fabricates $u_i \in \mathbb{G}$, $i \in Q$, and sends them to $\mathcal{C}$, who responds with $c_i \leftarrow_{\$} \mathbb{Z}_q$, $i \in Q$. Finally, $\mathcal{A}$ fabricates (s_i, y_i) , $i \in Q$, and wins only if (2.11) is satisfied. Note for later usage that the second part of this condition translates to

$$s_i = r_i + c_i x_i^*, \quad r_i \equiv \log u_i, \quad x_i^* \equiv \log y_i, \quad \forall i \in Q, \quad (2.2)$$

where no r_i 's or x_i^* 's can be assumed to be known to $\mathcal{A}$. In particular, we do not necessarily have $x_i^* = x_i$ for $i \in Q$, even though the first part of condition (2.11) holds true. Indeed, since y admits multiple (n, t) -sharings, $\mathcal{A}$ could possibly fabricate y_i 's that do not coincide with the public counterparts of x_i 's and yet interpolate to y . In fact, this is exactly what must be shown to be impossible.

Consider now an adversary $\mathcal{A}^*$ attacking (OM)DL while playing challenger to $\mathcal{A}$, to which it has forkable black-box access. That is, when given $y^* \equiv \exp(g, x^*)$, $x^* \leftarrow_{\$} \mathbb{Z}_q$, by its own challenger, $\mathcal{A}^*$ samples $x_i \leftarrow_{\$} \mathbb{Z}_q$, $i \in J$, and forwards $\{x_i\}_{i \in J}$ along with y to $\mathcal{A}$, where

$$y \equiv \prod_{i \in Q} y_i^{\lambda_i}, \quad y_i \equiv g^{x_i}, \quad i \in J, \quad y_j \equiv y^*.$$

Observe that $\mathcal{A}^*$ perfectly simulates $\mathcal{C}$ (cf. Lemma 5.2), so that $\mathcal{A}$ succeeds in its impersonation attempt with non-negligible advantage. The fork and extraction argument goes that $\mathcal{A}^*$

should be able to fork $\mathcal{A}$ at some special point, let it complete a second successful attempt with some non-negligible probability and then solve (OM)DL by extracting $x^* = x_j \equiv \log y_j$ from the two partially differing transcripts. However, since no assumptions can be made about $\mathcal{A}$'s strategy, equations (2.2) cannot be correlated with each other during this process, which means that the extraction formula can involve at most one of them. Consequently, forking at any location $i \in Q$ would at best yield x_i^* which –as previously stressed– cannot be assumed to coincide with x_i ; in particular, the process does not predictably yield the desired unknown x_j . We would need the individual proofs to be further joined into an atomic operation in order for the extraction of x_j to become possible.

2.11.1 Direct proof interpolation

Referring to the previous protocol, observe that every valid public share y_i is determined by u_i and s_i in the form $y_i = \sqrt[c_i]{g^{s_i}/u_i}$, $c_i \equiv H(u_i)$. This implies the following modification. Instead of attaching their public shares, the shareholders send ordinary Schnorr proofs for their respective secrets; upon aggregating $\{(u_i, s_i)\}_{i \in Q}$, the verifier skips individual knowledge verifications and proceeds to verify the interpolation condition

$$y = \prod_{i \in Q} \left(\frac{g^{s_i}}{u_i} \right)^{\lambda_i/c_i}. \quad (2.3)$$

Security against impersonation attacks should mean that every efficient adversary remains unable to fabricate a collection of proofs $\{(u_i, s_i)\}_{i \in Q}$ that interpolate to y in the above sense without knowing at least t secret shares. This intuition generalizes the Schnorr proof-of-knowledge property to the threshold setting and motivates the protocol introduced in the next section. Indeed, by completing the exponents in order to derive a more convenient expression, condition (2.3) reformulates to (3.3).

3 One-Step Schnorr Threshold (OSST) Identification

Fix a cyclic group $\mathbb{G} = \langle g \rangle$ of order $q \geq 1$ and a hash function $H : \mathbb{G} \rightarrow \mathbb{Z}_q$. Shamir's sharing is denoted by D (cf. Appendix B) with its parameters inferred from context.

3.1 The OSST protocol

Given integers (n, t) such that $1 \leq t \leq n < q$, we define the following scheme.

Protocol 3.1. (OSST _{$\mathbb{G}, n, t$} – *One-Step Schnorr* (n, t) -Threshold)

- *Setup.* Some uniformly random secret is distributed among a group of shareholders by means of Shamir's (n, t) -sharing, i.e.,

$$(x_1, \dots, x_n; y) \leftarrow D(g, x), \quad x \leftarrow_{\$} \mathbb{Z}_q$$

The threshold parameter and the public shares may remain private to the group, which needs only advertise the combined public key y .

- *Proving phase.* A coalition of shareholders $Q \subset \{1, \dots, n\}$ generate

$$(u_i, s_i), \quad s_i \leftarrow r_i + c_i x_i, \quad c_i \leftarrow H(u_i), \quad u_i \leftarrow g^{r_i}, \quad r_i \leftarrow_{\$} \mathbb{Z}_q \quad (3.2)$$

respectively and send them to the verifier, $i \in Q$.

- *Verification.* After aggregating (u_i, s_i) , $i \in Q$, the verifier accepts only if

$$\exp\left(g, \sum_{i \in Q} \mu_i s_i\right) = y^{\bar{c}} \prod_{i \in Q} u_i^{\mu_i}, \quad (3.3)$$

where

$$\bar{c} \leftarrow \prod_{i \in Q} c_i, \quad \mu_i \leftarrow \lambda_i \prod_{j \in Q \setminus \{i\}} c_j, \quad c_i \leftarrow H(u_i) \quad (3.4)$$

and $\{\lambda_i\}_{i \in Q}$ are the Lagrange interpolation coefficients for Q .

Remark 3.5. (*Validity condition*) By equating exponents, (3.3) translates to

$$\sum_{i \in Q} \mu_i (s_i - r_i) = \bar{c} \cdot x, \quad r_i \equiv \log u_i. \quad (3.6)$$

Definition 3.7. The *weights* and *normalizer* of a collection $\{u_i\}_{i \in Q} \subset \mathbb{G}$, $Q \subset \{1, \dots, n\}$, are $\{\mu_i\}_{i \in Q}$ and $\bar{c}$ respectively as defined in (3.4).

Remark 3.8. (*Relation between weights and the normalizer*) Let $\{\mu_i\}_{i \in Q}$, $\bar{c}$ be the weights and the normalizer of a collection $\{u_i\}_{i \in Q}$ respectively. Then

$$c_i \mu_i = \lambda_i \bar{c}, \quad \forall i \in Q \quad (3.9)$$

Remark 3.10. (*Correctness*) Let $y_i = g^{x_i}$, $1 \leq i \leq n$ be the public shares. Applying (3.9), the left-hand side of condition (3.3) transforms as

$$\prod_{i \in Q} g^{s_i \mu_i} = \prod_{i \in Q} g^{\mu_i c_i x_i} g^{\mu_i r_i} = \prod_{i \in Q} g^{x_i \lambda_i \bar{c}} \prod_{i \in Q} g_i^{r_i \mu_i} = \left(\prod_{i \in Q} y_i^{\lambda_i} \right)^{\bar{c}} \prod_{i \in Q} u_i^{\mu_i},$$

which yields the right-hand side if (and only if) $|Q| \geq t$. In other words, the proper execution of the protocol verifies if (and only if) at least t shareholders are involved.

Remark 3.11. (*Public-share inference*) Although these are not advertised, the verifier (or any man-in-the middle) can trivially infer the involved shareholders' public keys in the form

$$y_i = \sqrt[c_i]{\frac{g^{s_i}}{u_i}}, \quad i \in Q, \quad (3.12)$$

from $\{(u_i, s_i)\}_{i \in Q}$ respectively, since these are just ordinary Schnorr proofs for the corresponding secrets (cf. (3.2)). Note that provers are free to use any sharing of $x = \log y$ they want, be it “revoked”, “active” or with a different threshold. Successful identification is not bound to a specific sharing, since the verifier cannot know the public shares in advance and keep track of the sharing epochs; it only ensures that the provers have used some sharing consistently (provided that the protocol is secure against impersonation in the sense of Section 2.4). The onboarded entities are not committed to their inferred public keys, except maybe for the purpose of session-scoped accountability (cf. Section 3.3.3).

3.1.1 Interactive variant

We may regard H as a random oracle so long as predicting any of its output bits is infeasible. That is, the provers send their commitments to the verifier, who lazily samples challenges by maintaining a lookup table Map per session. As usual, we will employ this variant in order to provide a formal security proof in the random oracle model.

More accurately, the i -th prover samples $r_i \leftarrow_{\$} \mathbb{Z}_q$ and sends $u_i \leftarrow g^{r_i}$ to the verifier. If $u_i \notin \text{Dom}(\text{Map})$, the verifier samples $c_i \leftarrow_{\$} \mathbb{Z}_q$ and caches $\text{Map}[u_i] \leftarrow c_i$; in either case, it sends $c_i \leftarrow \text{Map}[u_i]$ to the prover, who responds with $s_i \leftarrow r_i + c_i x_i$. After aggregating s_i , $i \in Q$, the verifier checks condition (3.3) using the previously cached challenges c_i , $i \in Q$.

3.2 Practical efficiency

The provers behave as in the non-distributed protocol, sending asynchronously their offline generated proofs to the verifier; they remain unaffected by network overhead because the proving phase is non-interactive. That is, apart from the inherent delays of asynchronicity and packet transmission, system latency depends entirely on the performance of the verifier.

OSST verification reformulates condition (2.3), which requires $2k$ exponentiations (where $k = |Q|$) without being amenable to optimizations; even applying multi-exponentiation across the product, the verifier would have to compute the nested elements $g^{s_i u_i^{-1}}$, $i \in Q$, via k separate exponentiations. By contrast, (3.3) reduces the number of exponentiations to $k + 2$ and –most importantly– its right-hand side is directly amenable to multi-exponentiation.

Under naive exponentiation, the conservative calculation in Section 6 estimates total verification time at ~ 40 ms for $k = 100$ and ~ 500 ms for $k = 1000$ using NIST P-256 on an older x86-64 processor (2011) with 3.5 GHz clock rate (cf. Table 5), provided that the weights are computed efficiently (cf. Section 6.4). When the same verifier employs wNAF-based multi-exponentiation (cf. Section 6.2.1), the total execution time falls to ~ 7 -12 ms for $k = 100$ (cf. Tables 6) at a peak memory penalty of ~ 20 -160 KB (including the storage need for the precomputed elements).

3.2.1 Implementation challenges

No peer-to-peer software needs to be implemented because the provers operate asynchronously as in the ordinary Schnorr protocol; similarly, no low-latency overlay network needs to be established and maintained between them because the protocol is non-interactive on their side. The implementation effort concentrates on the verifier, i.e., the secure software required for verifying condition (3.3), and its associated optimizations. Note that, while well-studied and widely implemented, multi-exponentiation algorithms require per device profiling and tuning, so that the tradeoff between window size and runtime remains effective under the specific hardware constraints of the verifier (including the capacity of its CPU cache). This is a local optimization problem, potentially addressable by vertical scaling, whose solution does not require continuous governance and is more tractable than tuning the global parameters of an overlay network. An indicative optimization roadmap is provided in Section 6.

Beyond algebraic optimizations, OSST verifiers should be thought of as state machines that have to address several interconnected issues related to session management. These include the exact aggregation model for the incoming packets, the scheduling of incremental verification (cf. Section 3.3.1) and their defense mechanisms against session poisoning (cf. Section 3.4.2).

3.3 Operational considerations

Below we examine OSST from the operability and interoperability viewpoint. In Section 3.3.1 we show how it achieves incremental verification during its stable phase. In Section 3.3.2 we show how that same mechanism facilitates constant-time verification, allowing for predictable latencies and controlled system efficiency after a baseline coalition has been detected. In Section 3.3.3 we describe how OSST decouples accountability from individual authentication, effectively serving itself as a share attestation mechanism. In Section 3.3.4 we analyze its failure to qualify as a practical signature scheme and determine the space cost of verifiable audit trails for logged identification events.

3.3.1 Incremental verification

Being non-interactive on the provers' side, the protocol facilitates incremental verification (cf. Section 2.3.2) allowing the provers to join and leave dynamically. Concretely, if the verifier retains the aggregated packets $\{(u_i, s_i)\}_{i \in Q}$ received from an already identified coalition Q , $|Q| \geq t$, any shareholder $j \notin Q$ can attach itself asynchronously to the current session without requiring Q to rerun the proving phase. This is because the verifier can simply recompute (3.3) for the augmented set $Q^* \equiv Q \cup \{j\}$ in order to ensure that packet (u_j, s_j) preserves the validity condition. In this context, Q will be referred to as a *baseline coalition*. Detecting a baseline coalition is crucial for every OSST session; at that point, the session enters its stable phase, where verification can be constant-time (cf. Section 3.3.2) and the session itself becomes immune to poisoning (cf. Section 3.4.2).

Various strategies are here possible for reducing the costs of repeated verification. For example, the verifier could exploit recursive relations between the weights of Q^* and Q to decrease the number of field operations per incoming connection; however, this cannot generally avoid k modular inversions (a relatively expensive operation), meaning that the gain would only appear for large k 's and be marginal. Assuming that weights can always be computed from scratch efficiently (cf. Section 6.4), wNAF multi-exponentiation is particularly tailored to the needs of incremental verification due to its decoupled precomputation strategy (cf. Section 6.2.1). The exact loop and its associated cost model are described in Section 6.6.

3.3.2 Constant-time verification

The verifier response time is generally unpredictable due to the asynchronous nature of the protocol. Specifically, since provers operate independently at their own pace, the verifier has to defer its responses until a baseline coalition is detected among the continuously aggregated proof packets. In the extreme case where no session timeout is applied by the verifier and a baseline coalition never identifies, the individual provers have to await indefinitely.

The situation is different after some baseline coalition Q has been identified. During this phase, when a shareholder sends its proof to the verifier, the latter's response time is determined by the time required for processing it in the backend; no random costs occur from asynchronicity and –aside from packet transmission– latency is that of incremental verification (cf. Section 3.3.1). This evidently depends on $k = |Q|$ (cf. Section 6.6), meaning that response time grows with the number of onboarded entities if the baseline coalition is incrementally augmented with them. We can avoid this by verifying incoming proof packets against a fixed baseline coalition of size $|Q| = t$; in this case, verification time becomes a system-wide constant and protocol efficiency stabilizes around its value. Side-channel attacks cannot exploit latency to infer the number of onboarded shareholders.

3.3.3 Session-scoped accountability – Combiner as-a-service

Even if individual identities remain opaque in their output, threshold cryptographic operations can take place in a permissioned environment where the shareholders participate accountably. For instance, in a threshold signature scheme, the combiner typically validates the partial signatures before combining them and forwarding their result to the outer world; similarly, during threshold decryption, the combiner must validate each partial decryptor against the respective share in order to ensure its consistency with the ciphertext. In general, the combiner must know and trust the public shares in advance; since the received packets typically contain an embedded proof-of-knowledge for the corresponding secret key, these simultaneously serve as means to authenticate each shareholder individually, thereby granting them access to the permissioned environment where the combiner offers its services. This is an implicit application of the trivially decoupled protocol described in Section 2.10.2.

In closed committees (i.e., if the combiner is a shareholder), trust is established during group constitution (cf. Section 2.1.1); this is the case of a financial consortium that maintains a combiner for outputting its transaction signatures. If the combiner is external to the group, it must still belong to a closed organization where trusted entities attest to the correctness of the advertised shares, while the system itself must be ephemeral to avoid the complications of proactive sharing. This is the case of an election server that combines the partial decryptors received from the board of trustees to decrypt the collected votes. In open environments, neither of these approaches is practical and the feasibility of a publicly available combiner is bounded by the impracticality of universal share certification mechanisms (cf. Section 2.2.1); the combiner cannot be deployed as-a-service unless able to enforce its custom certification policy on each registered group and further monitor the lifecycle of the individual member identities. Interoperability is decreased while the attack surface becomes unmanageable.

Nevertheless, onboarding a qualified coalition of shareholders to a permissioned environment *and* holding them accountable for their activity can sometimes be a weaker constraint than authenticating them individually against their already trusted shares; while a trusted share mechanism ensures both conditions at once, these can probably be decoupled and satisfied by other means, especially if accountability is an ephemeral session-scoped requirement. Indeed, the onboarding step is the objective of any real-time threshold identification protocol while subsequent accountability is facilitated automatically if the involved public shares can be inferred on the fly from its transcript. We proceed to see how OSST fits into this scheme.

Let y be a group public key that has been registered on a combiner service $\mathcal{C}$ (e.g., a threshold signature gateway or delegated decryption service). When a coalition of shareholders request access to it, $\mathcal{C}$ executes OSST in the role of verifier in order to identify their ensemble over the respective communication channels and thus onboard them to its permissioned environment. At this point, $\mathcal{C}$ does not yet know the onboarded entities' public shares and consequently cannot hold them cryptographically accountable for their actions. However, $\mathcal{C}$ can infer the shares in the form (3.12) and use them to validate any subsequent activity during the current session. It does not need to know and trust their fixed shares in advance.

This shows how OSST allows combiners to operate in open settings without relying on pre-validations. The consortium of signers does not need to maintain an internal combiner for constructing their threshold signature; they can use an external API, provided that their group public key has been registered on it. They may not even need to maintain a peer-to-peer network if the combiner can operate as a relay node between them. Conversely, the combiner does not need to certify the consistency and correctness of any individual member identities or monitor their refresh; it needs only “trust” once the entity that registers the group public key (e.g., because it “pays” registration “fees”). It can be deployed conveniently as-a-service.

3.3.4 Inefficient signing primitive

A multi-party signature scheme is considered to be “practical” if its output components combine into an aggregate signature that cannot be distinguished from the output of the non-distributed case; in particular, the combined signature should be verifiable against the combined public key in the ordinary fashion. This signature can be efficiently stored and transferred for subsequent verifications and ordinary verifiers can be employed as plug-and-play components in the distributed setting. If at the cost of interactivity on the provers’ side (cf. Section 2.3.1), this property is crucial for distributed ledgers because it ensures backward compatibility with existing transaction formats, interoperability with the single-signer model and scalability from the computational perspective.

Building upon the Fiat-Shamir transform, OSST turns into a threshold signature scheme by letting the provers bake messages into their respective challenges. We will not investigate unforgeability, focusing on the mechanics of the derived signature scheme as well as its failure to be practical in the previous sense. Referring to the notation of Protocol (3.1), let $\mathcal{M}$ be a message space and the hash function be of the form $H : \mathbb{G} \times \mathcal{M} \rightarrow \mathbb{Z}_q$. Given respective messages $m_i \in \mathcal{M}$, $i \in Q$, the coalition of shareholders $Q \subset \{1, \dots, n\}$ generate

$$(m_i, (u_i, s_i)), \quad s_i \leftarrow r_i + c_i x_i, \quad c_i \leftarrow H(u_i, m_i), \quad u_i \leftarrow g^{r_i}, \quad r_i \leftarrow_{\mathbb{S}} \mathbb{Z}_q, \quad i \in Q,$$

and send them to the combiner; after collecting the signed messages $\{(m_i, (u_i, s_i))\}_{i \in Q}$, the combiner accepts only if the condition

$$g^{\bar{s}} = y^{\bar{c}} \prod_{i \in Q} u_i^{\mu_i}, \quad \bar{s} \equiv \sum_{i \in Q} \mu_i s_i, \tag{3.13}$$

is satisfied, where $c_i \leftarrow H(u_i, m_i)$, $i \in Q$. Clearly, in order for verification to be reproducible, the combiner must store the aggregate signature $(\bar{s}, \{u_i\}_{i \in Q})$, which grows linearly in $|Q|$. That is, we can dispense with $\{s_i\}_{i \in Q}$ in favour of $\bar{s}$ but we cannot discard the individual commitments $\{u_i\}_{i \in Q}$ because the hashed messages remain uncoupled. The ordinary verifier cannot be used as is in the distributed setting.

Non-compactness Irrespective of verifier pluggability, deriving space-efficient signatures can be important when identification events must be loggable as such in a verifiable fashion. For example, in systems where critical one-off operations are released by means of distributed authorization, the provers might be asked to bake messages into their packets for the purpose of validating a chain of events, failure diagnostics and post-mortem investigations. If $|Q| \geq t$ is not excessively large, $(\bar{s}, \{u_i\}_{i \in Q})$ can be an efficient audit trail that preserves the advantage of being non-interactive; if asynchronicity is a hard requirement, then this is probably a best effort solution with respect to both storage and computational efficiency. Indeed, there is growing consensus that any Schnorr-based distributed protocol with constant-size output must necessarily be interactive on the provers’ side (e.g., [34], [22], [7]) due to the absence of homomorphic aggregability, which forces the signers to engage in multi-party computations in order to collectively generate the challenge; in this regard, OSST may represent the highest degree of compactification possible so long as the signers do not interact with each other.⁶

⁶ Aggregate signatures that are both compact and non-interactive are possible in bilinear cryptography (namely the BLS scheme [10]), which facilitates homomorphic aggregability outside the pure DL setting.

3.4 Security considerations

Since OSST decouples into independent Schnorr proofs, any security precaution regarding the latter applies to the former. For example, the protocol remains susceptible to side-channel attacks if the random number generator leaks linear relations between the discrete logarithms of distinct commitments; similarly, choosing any of the challenges to be zero causes the verifier to accept trivially. Below we focus on high-level attacks that arise particularly in the distributed setting.

3.4.1 Generalized replay attack

OSST is as much susceptible to replay attacks as the ordinary Schnorr protocol; a man-in-the-middle can passively capture and replay the packets sent by a coalition of provers in order to impersonate them anytime. Moreover, the attacker has $t - 1$ degrees of freedom in modifying the response values $\{s_i\}_i$ while keeping $\{u_i\}_i$ fixed. Let $\{(u_i, s_i)\}_{i \in Q}$ be a valid proof with $|Q| \geq t$ and weights $\{\mu_i\}_{i \in Q}$. For fixed $j \in Q$, pick arbitrary $\{\delta_i\}_{i \in J} \subset \mathbb{Z}_q$ for $J = Q \setminus \{j\}$ and define

$$\delta_j = -\frac{1}{\mu_j} \sum_{i \in J} \mu_i \delta_i.$$

Then $\{(u_i, s_i^*)\}_{i \in Q}$ with $s_i^* = s_i + \delta_i$ remains valid because

$$\sum_{i \in Q} \mu_i s_i^* = \sum_{i \in Q} \mu_i s_i + \left(\mu_j \delta_j + \sum_{i \in J} \mu_i \delta_i \right) = \sum_{i \in Q} \mu_i s_i$$

(cf. Corollary 3.29). This essentially exploits the structure described in Remark 3.32 and is akin to a multi-round execution (cf. Section 3.5.1). Observe that this generalizes the replay attack of the non-distributed case $n = t = 1$, where the intercepted proof can only be replayed as is. Generalized replay attacks can be mitigated by the same method as in the non-distributed case, i.e., by baking (distinct) nonces into the challenges in order to maintain (separate) states between the provers and the verifier at the application layer.

3.4.2 Session poisoning – Reactive fault detection

Like all share-free thresholds, OSST is susceptible to the DoS attack of Section 2.6, obstructing a legitimate coalition $Q \subset \{1, \dots, n\}$, $|Q| \geq t$, from being identified. Concretely, the attacker submits a structurally valid but semantically incorrect proof packet

$$(u_j, s_j), \quad u_j \leftarrow_{\$} \mathbb{G}, \quad s_j \leftarrow_{\$} \mathbb{Z}_q, \quad j \in \{1, \dots, n\} \setminus Q$$

to the verifier; in case of bulk verification, the coalition $Q^* = Q \cup \{j\}$ fails to identify in the sense of (3.3) and service is thus denied to Q . Nevertheless, reactive fault detection facilitates immunity to poisoning after a baseline coalition has been identified; consequently, it is the verifier's priority to detect such a coalition as early as possible within a pool of initially aggregated proof packets. The exact scheduling belongs to the specific use case and the hardware capabilities of the verifier; we here remark that it can benefit greatly from the precomputation strategy of wNAF-based verification (cf. Section 6.5.2) and the parallel invocation of its main evaluation stage across separate threads. In any case, uncritically exhausting combinations can drain the verifier's resources in case of flooding and must be prevented via rate limiting or CPU throttling.

3.4.3 Key-recovery attack – Interactive aspect

This is a cryptanalytic attack affecting the interactive variant of the protocol (cf. Section 3.1.1) under rather special circumstances but sheds light on its inherent security structure. Similar to the non-distributed case, we will employ this attack in order to derive a formal security proof by means of fork and extraction (cf. Section 5.4). Evidently, if a shareholder uses the same commitment in two canonical executions of the interactive OSST, then its secret key leaks trivially as in the non-distributed case. This is ordinary key-recovery embedded into the canonical threshold setting. We will show that key-recovery is possible in the context of multi-round executions (cf. Section 3.5.1) in the presence of a malicious verifier who controls $t - 1$ shareholders.

Let $Q = J \cup \{j\}$, where $|J| = t - 1$ with $j \notin J$, be a coalition of shareholders and the cluster J be controlled by an adversary $\mathcal{A}^*$. We further assume that $\mathcal{A}^*$ controls the verifier. The shareholders engage in a multi-round execution and generate a transcript $\{(u_i, c_i, s_i)\}_{i \in Q}$ that verifies, so that

$$\sum_{i \in Q} \mu_i (s_i - r_i - c_i x_i) = 0$$

holds true, where $r_i \equiv \log u_i$ and $\{\mu_i\}_{i \in Q}$ are the weights induced by $\{c_i\}_{i \in Q}$ (cf. Cor. 3.29). They next engage in a second execution, where the j -th shareholder reuses u_j . After receiving u_j on the server's side, $\mathcal{A}^*$ responds with a challenge $c_j^* \neq c_j$ and tunes the rest parties to reuse the commitments u_i , $i \in J$, from the previous session. To each of them, the server responds with the same challenge $c_i^* = c_i$ from the previous session. Subsequently, the provers compute honestly their responses s_i , $i \in Q$, and send them to the server; since the transcript $\{(u_i, c_i^*, s_i^*)\}_{i \in Q}$ verifies, we again have

$$\sum_{i \in Q} \mu_i^* (s_i^* - r_i - c_i^* x_i) = 0,$$

where $\{\mu_i^*\}_{i \in Q}$ are the weights induced by $\{c_i^*\}_{i \in Q}$. Note that $c_i = c_i^*$ for $i \neq j$, so that $\mu_j = \mu_j^*$ (cf. Definition 3.7). Subtracting terms and applying this remark, we get

$$\mu_j (s_j - s_j^* - (c_j - c_j^*) x_j) + \sum_{i \in J} (\mu_i (s_i - r_i - c_i x_i) - \mu_i^* (s_i^* - r_i - c_i x_i)) = 0.$$

This eliminates r_j , which is the only unknown to $\mathcal{A}^*$. Since $c_j - c_j^* \neq 0$, the adversary can use this relation to retrieve x_j in the form

$$x_j = \frac{s_j - s_j^*}{c_j - c_j^*} + \frac{1}{\mu_j} \sum_{i \in J} (\mu_i (s_i - r_i - c_i x_i) - \mu_i^* (s_i^* - r_i - c_i x_i)). \quad (3.14)$$

For $n = 1$, this reduces to the usual extraction formula for the non-distributed protocol.

3.5 Validity conditions

In this section, we provide characterizations of validity in the sense of condition (3.3).

Definition 3.15. A *proof* is a collection $\sigma = \{(u_i, s_i)\}_{i \in Q}$ of the form

$$(u_i, s_i) \in \mathbb{G} \times \mathbb{Z}_q, \forall i \in Q \wedge Q \subset \{1, \dots, n\}.$$

Given σ as above, we refer to $\{u_i\}_{i \in Q}$ as its *commitments* and say that σ is *based on* them. We also refer to $\{s_i\}_{i \in Q}$ as the *responses*. The *weights* and *normalizer* of σ are the weights and normalizer of its commitments (cf. Def. 3.7) respectively. The *proof space* is the set of proofs and will be denoted by Σ . We also denote by

$$\Sigma[\{u_i\}_{i \in Q}]$$

the subspace of proofs that are based on $\{u_i\}_{i \in Q}$.

The proper execution of the protocol is modelled as follows.

Definition 3.16. The proof $\{(u_i, s_i)\}_{i \in Q} \in \Sigma$ is called *canonical with respect to* a sharing $(x_1, \dots, x_n; y) \in D_x$ (cf. Def. B.2) if

$$s_i = r_i + c_i x_i, \quad c_i \equiv H(u_i), \quad r_i \equiv \log u_i, \quad \forall i \in Q, \quad (3.17)$$

i.e., its components are Schnorr proofs for the respective shares. Evidently, the subspace $\Sigma[\{u_i\}_{i \in Q}]$ contains a unique canonical representative with respect to every fixed sharing.

Definition 3.18. The *one-step Schnorr* (n, t) -*threshold verifier* is the deterministic algorithm $\mathcal{V} : \mathbb{G} \times \Sigma \rightarrow \{\text{true}, \text{false}\}$ invoked as

$$(y, \{(u_i, s_i)\}_{i \in Q}) \mapsto \exp\left(g, \sum_{i \in Q} \mu_i s_i\right) = y^{\bar{c}} \prod_{i \in Q} u_i^{\mu_i},$$

where $\{\mu_i\}_{i \in Q}$, $\bar{c}$ are the weights and the normalizer of $\{(u_i, s_i)\}_{i \in Q}$ respectively.

Definition 3.19. A proof $\sigma \in \Sigma$ is called *valid against* $y \in \mathbb{G}$ if $\mathcal{V}(y, \sigma) = \text{true}$.

OSST correctness (cf. Rem. 3.10) means that a proof canonical with respect to $(x_1, \dots, x_n; y)$ is valid against y if and only if $|Q| \geq t$. The subspace of proofs that are based on $\{u_i\}_{i \in Q}$ and valid against y will be denoted by

$$\Sigma_y[\{u_i\}_{i \in Q}].$$

If $|Q| \geq t$, then this space contains a unique canonical representative with respect to every fixed sharing of $x = \log y$. In the non-distributed case $n = 1$, the subspace $\Sigma_y[\{u_i\}_{i \in Q}]$ is a singleton due to the “unique responses” property of the Schnorr protocol.⁷ In general, however, this space inflates at a rate dependent on $|Q| \geq t$ (cf. Remarks 3.25 and 3.32), implying an abundance of valid proofs that are not canonical.

Proposition 3.20. (*Validity conditions*) Let $x \in \mathbb{Z}_q$, $y = g^x$ and $\sigma = \{(u_i, s_i)\}_{i \in Q}$. Then, with overwhelming probability, σ is valid against y if and only if

⁷ For every u there exists a unique s such that the proof (u, s) is valid against $y = g^x$, namely $s = r + H(u)x$ with $r \equiv \log u$.

$$\frac{1}{\bar{c}} \sum_{i \in Q} \mu_i (s_i - r_i) = x, \quad r_i \equiv \log u_i, \quad (3.21)$$

where $\{\mu_i\}_{i \in Q}$, $\bar{c}$ are the weights and normalizer of σ respectively, or, equivalently,

$$x = \sum_{i \in Q} \lambda_i \frac{s_i - r_i}{c_i}, \quad (3.22)$$

where $c_i \equiv H(u_i)$ and $\{\lambda_i\}_{i \in Q}$ are the interpolation coefficients for Q .

Proof. Since H is a hash function, $\bar{c} \neq 0$ with overwhelming probability in the bitlength of q . In this case, the validity condition (3.6) transforms to (3.21). This further reformulates to (3.22) by means of (3.9). $\square$

This statement holds true irrespective of size (including the case $|Q| < t$), establishing a relation between x and any proof that happens to be valid against y . Both (3.21) and (3.22) generalize the relation of a secret key to a valid non-distributed proof; it should be stressed however that (3.22) does not imply $x_i = (r_i - s_i)/c_i$ and it can hold true without σ being canonical. Indeed (cf. Remark 3.25), valid proofs do not confine to those properly generated and a relevant security notion should ensure that such proofs are infeasible to compute without knowledge of at least t secret shares (cf. Section 4). The exact meaning of (3.22) is clarified in Proposition 3.24. Evidently, a proof should not verify against different public keys. Indeed, this key almost always exists and it is unique.

Proposition 3.23. *(Almost every proof is valid against some unique public key) For any commitments $\{u_i\}_{i \in Q} \subset \mathbb{G}$, with overwhelming probability, every $\sigma \in \Sigma$ based on them is valid against some unique $y \in \mathbb{G}$.*

Proof. Let $\{\mu_i\}_{i \in Q}$, $\bar{c}$ be the weights and normalizer of $\{u_i\}_{i \in Q}$ respectively. Since H is a hash function, $\bar{c} \neq 0$ with overwhelming probability in the bitlength of q . Consequently, given $\sigma = \{(u_i, s_i)\}_{i \in Q}$, we can define

$$x = \frac{1}{\bar{c}} \sum_{i \in Q} \mu_i (s_i - r_i), \quad r_i \equiv \log u_i$$

By Proposition 3.20, σ is valid against $y = g^x$ solely. $\square$

Proposition 3.24. *(Almost every proof of size $|Q| = t$ is canonical with respect to some unique sharing) Given commitments $\{u_i\}_{i \in Q} \subset \mathbb{G}$ with $|Q| = t$, then, with overwhelming probability, every $\sigma \in \Sigma$ based on them is canonical with respect to some unique sharing $(x_1, \dots, x_n; y) \in D_x$ of some unique secret $x \in \mathbb{Z}_q$.*

Proof. Let $\{\mu_i\}_{i \in Q}$ be the weights and $c_i \equiv H(u_i)$, these parameters being common to all proofs based on $\{u_i\}_{i \in Q}$. Since H is a hash function, $c_i \neq 0, \forall i \in Q$, with overwhelming probability in the bitlength of q . Consequently, given a proof $\sigma = \{(u_i, s_i)\}_{i \in Q}$, we can define

$$x_i = \frac{s_i - r_i}{c_i}, \quad r_i \equiv \log u_i, \quad i \in Q.$$

Since $|Q| = t$, the scalars $\{x_i\}_{i \in Q}$ uniquely determine a (n, t) -sharing $(x_1, \dots, x_n; y)$ of some $x \in \mathbb{Z}_q$. Since $s_i = r_i + c_i x_i, \forall i \in Q$, the proof is canonical with respect to this sharing and thus valid against $y = g^x$. The claim follows from Proposition 3.23. $\square$

Remark 3.25. (*Validity and degrees of freedom*) Fix commitments $\{u_i\}_{i \in Q} \subset \mathbb{G}$. Choose $J \subset Q$ with $|J| = |Q| - 1$, pick arbitrary $\{s_i\}_{i \in J} \subset \mathbb{Z}_q$ and set

$$s_j = r_j + \frac{1}{\mu_j} \left(\bar{c} \cdot x - \sum_{i \in J} \mu_i (s_i - r_i) \right)$$

where $\{j\} = Q \setminus J$ and $r_i \equiv \log u_i$, $i \in Q$. By Proposition 3.20, this process exhausts the proofs that are based on $\{u_i\}_{i \in Q}$ and valid against $y \equiv g^x$ bijectively. That is, the subspace $\Sigma_y[\{u_i\}_{i \in Q}]$ has $|Q| - 1$ degrees of freedom in the sense that every proof is uniquely determined by the choice of $\{s_i\}_{i \in J}$ and $J \subset Q$ with $|J| = t - 1$ (For $|Q| = 1$ this yields the “unique responses” property of the ordinary Schnorr protocol). Observe that this process has no obvious utility for an impersonating adversary, since the latter would still need to eliminate the combined secret x in order to compute s_j .

Proposition 3.26. (*Validity against sharing*) Let $x \in \mathbb{Z}_q$ and $\sigma = \{(u_i, s_i)\}_{i \in Q} \in \Sigma$. Given $(x_1, \dots, x_n; y) \in D_x$, the proof σ is valid against y if and only if

$$\sum_{i \in Q} \mu_i (s_i - r_i - c_i x_i) = \bar{c} \cdot \left(x - \sum_{i \in Q} \lambda_i x_i \right), \quad (3.27)$$

Proof. Follows from the verification condition (3.6) by subtracting

$$\sum_{i \in Q} \mu_i c_i x_i$$

from both sides and applying (3.9) on the right. $\square$

Remark 3.28. (*Valid proofs with $|Q| < t$*) We know that if a proof of size $< t$ is valid, then it cannot be canonical (cf. Remark 3.10). Such proofs exist in abundance (cf. Remark 3.25) and Proposition 3.26 indicates a way to generate them programmatically. Fix $\{u_i\}_{i \in Q} \subset \mathbb{G}$ and set $s_i = r_i + c_i x_i$, where $r_i \equiv \log u_i$, $c_i \equiv H(u_i)$. Pick $S \subset Q$ with $S \neq \emptyset$ and define $s_i^* = s_i$ if $i \notin S$, otherwise

$$s_i^* = s_i + \delta_i, \quad \delta_i \equiv \frac{1}{\mu_i} \cdot \frac{\bar{c}}{|S|} \cdot \left(x - \sum_{j \in Q} \lambda_j x_j \right),$$

where $\bar{c}$ is the normalizer of $\{u_i\}_{i \in Q}$. Consider the proof $\sigma = \{(u_i, s_i^*)\}_{i \in Q}$. If $|Q| \geq t$, then $\delta_i = 0$ for all $i \in S$, so that σ is canonical and thus valid. If $|Q| < t$, then σ is still valid because it satisfies condition (3.27) by design. Note however that this construction has no obvious utility for a malicious coalition $Q \subset \{1, \dots, n\}$ with $|Q| < t$, as the colluders (or an impersonating adversary who knows $\{x_i\}_{i \in Q}$) would still need to know the combined secret x in order to adjust the δ_i ’s appropriately.

Corollary 3.29. (*Validity against sharing, $|Q| \geq t$*) Let $x \in \mathbb{Z}_q$ and $\sigma = \{(u_i, s_i)\}_{i \in Q} \in \Sigma$ with $|Q| = t$. Given $(x_1, \dots, x_n; y) \in D_x$, the proof σ is valid against y if and only if

$$\sum_{i \in Q} \mu_i s_i = \sum_{i \in Q} \mu_i (r_i + c_i x_i), \quad (3.30)$$

where $r_i \equiv \log u_i$, $c_i \equiv H(u_i)$ and $\{\mu_i\}_{i \in Q}$ are the weights of σ .

Proof. The right-hand side of (3.27) vanishes exactly if $|Q| \geq t$. $\square$

Remark 3.31. ($t - 1$ degrees of freedom) Fix $\{u_i\}_{i \in Q}$ with $|Q| = t$. Choose $J \subset Q$ with $|J| = t - 1$, pick arbitrary $\{\delta_i\}_{i \in J} \subset \mathbb{Z}_q$ and set

$$s_j = r_j + c_j x_j - \frac{1}{\mu_j} \sum_{i \in J} \mu_i (s_i - r_i - c_i x_i),$$

where $\{j\} = Q \setminus J$ and $r_i \equiv \log u_i$, $c_i \equiv H(u_i)$. By Corollary 3.29, this process exhausts the proofs that are based on $\{u_i\}_{i \in Q}$ and valid against y bijectively. This is a special case of Remark 3.25 with reference to a fixed sharing. Observe that it has no obvious utility for an adversary who knows $\{x_i\}_{i \in J}$ but not x_j .

Remark 3.32. (*Valid proofs with $|Q| \geq t$*) Remark 3.31 can be seen from a more general perspective. Consider a canonical proof $\{(u_i, s_i)\}_{i \in Q}$ with $|Q| \geq t$ and let $\{\mu_i\}_{i \in Q}$ be the weights. Pick $j \in Q$ and δ_i , $i \in Q \setminus \{j\}$, such that at least one is non-zero. If we define

$$\delta_j = -\frac{1}{\mu_j} \sum_{i \neq j} \mu_i \delta_i,$$

then the proof $\{(u_i, s_i + \delta_i)\}_{i \in Q}$ satisfies condition (3.30) by design. Evidently, this process exhausts the valid proofs that are based on $\{u_i\}_{i \in Q}$ bijectively (cf. Corollary 3.29).

3.5.1 Multi-round executions

Assuming that traceable communication channels exist between them and the relevant network overhead is not prohibitive, any $|Q| \geq t$ provers can interactively generate a non-canonical proof without revealing their secret shares to each other. In particular, instead of directly sending the packets (u_i, s_i) , $i \in Q$, to the verifier, the provers can first engage in the multi-party computation of scalars $\{\delta_i\}_{i \in Q}$ such that

$$\sum_{i \in Q} \mu_i \delta_i = 0$$

and send the packets (u_i, s_i^*) , $i \in Q$, where $s_i^* = s_i + \delta_i$. Indeed, if $|Q| \geq t$ then the proof $\{(u_i, s_i^*)\}_{i \in Q}$ remains valid because

$$\sum_{i \in Q} \mu_i s_i^* = \sum_{i \in Q} \mu_i s_i + \sum_{i \in Q} \mu_i \delta_i = \sum_{i \in Q} \mu_i s_i,$$

cf. Corollary 3.29. By this same corollary, every valid proof of size $|Q| \geq t$ can be generated with a multi-round execution of the above kind.

Multi-round executions are important from the security analysis viewpoint because they model a wide range of impersonating strategies. This is because an adversary who controls $t - 1$ shareholders $J \subset \{1, \dots, n\}$ has increased chances to seduce a single shareholder $j \notin J$ into a multi-round execution and thus fabricate a valid proof. A procedure of this kind is indeed employed against the interactive variant of the protocol in order to recover the j -th secret share (cf. Section 3.4.3). In fact, Corollary 3.29 indicates that multi-round executions are the natural setting for deriving the threat model of the OSST protocol (cf. Section 4.1).

4 Security notions

In this section, we derive and specify the OSST threat model. Attack games are formulated in the asymptotic framework and negligibility is meant with respect to the bitlength of the group order. In Section 4.1.1, we outline security against impersonation attacks and correlate it with a pragmatic design requirement, namely a distributed knowledge property. In Section 4.1.2, we argue that this property cannot be derived under DL hardness alone, implying that security against impersonation attacks is most probably irreducible to the latter; nevertheless, we observe that the stronger OMDL assumption (cf. Appendix C) is sufficient for deriving the knowledge property or, equivalently, yields the missing premises for a security proof by means on fork and extraction. In Section 4.1.3 we examine some direct consequences of OMDL hardness and further narrow our attack model so that the sufficient conditions of extractability are satisfied (cf. Section 3.4.3). The working model for impersonation attacks is defined in Section 4.2.

4.1 Threat model

The current standard for identification protocols is security against active impersonation attacks. This refers to adversaries who not only intercept multiple identification sessions before mounting their actual attack, but can also engage in them actively by impersonating a legitimate verifier (e.g., by cloning a website in order to collect information about the user's credentials); after several such interactions, the adversary switches roles and attempts to identify itself against some verifier as a legitimate user. We will not pursue this path, focusing on passive adversaries who potentially exert some control over the network. Once the relation between threshold and ordinary identification has been clarified from the security viewpoint, immunity against active adversaries should become possible to establish using techniques similar to those of [8].

4.1.1 General attack model

In most general terms, the OSST threat model consists of an adversary $\mathcal{A}$ who knows the keys of up to $t - 1$ shareholders and attempts to impersonate some coalition $Q \subset \{1, \dots, n\}$, i.e., fabricate a proof $\{(u_i, s_i)\}_{i \in Q}$ that verifies. It does not make sense to consider adaptive adversaries because the proving phase consists of decoupled proofs asynchronously sent to the verifier; since the provers operate in one step independently, $\mathcal{A}$ has no chance to collect intermediate information and potentially use it in order to corrupt them adaptively. We can thus confine ourselves to the static model, assuming that the attacker decides from the outset the set of corrupted parties. Adaptive corruption during the execution of an equivalent DKG during the key setup phase belongs to the threat model of the particular use case.

One could further assume that $\mathcal{A}$ knows the involved shareholders' public keys; indeed, although not advertised, these can be trivially inferred from the protocol transcript (cf. Remark (3.11)). Letting the adversary know them from the outset corresponds to a scenario where the public shares become advertised within the group for at least some initial timespan (e.g., for mutual verification purposes) and $\mathcal{A}$ compromises the device of a shareholder who has not yet erased them. Still, the security proof of Section 5.5 remains intact (cf. Remark 5.16), implying that an adversary who controls devices in the non-erasure model has no greater advantage than one who only steals keys.

Let $(x_1, \dots, x_n; y)$ be the given (n, t) -sharing of some secret key x and $\mathcal{A}$ be an adversary who knows the secret shares $\{x_i\}_{i \in J}$ with $|J| = t - 1$. Let $Q \subset \{1, \dots, n\}$ be the shareholders

that fall victim to impersonation under a purportedly valid proof $\{(u_i, s_i)\}_{i \in Q}$ fabricated by $\mathcal{A}$. Note that, without loss of generality, $Q \subset J$ or $J \subset Q$. Indeed, since $\mathcal{A}$ knows nothing special about the uncorrupted parties, the choice of Q is irrelevant as far as Q intersects with $\{1, \dots, n\} \setminus J$; we can let Q fluctuate without narrowing the threat model, provided that its size $|Q|$ is preserved and the original intersection with J remains included. We conclude with the following general attack model.

Game 4.1. Given adversary $\mathcal{A}$ and challenger $\mathcal{C}$,

- *Attack statement and corruption:*
 - $\mathcal{A}$ chooses $J \subset \{1, \dots, n\}$ with $|J| = t - 1$.
 - $\mathcal{A}$ chooses $Q \subset \{1, \dots, n\}$ with $Q \subset J$ or $J \subset Q$ and sends it along with J to its challenger $\mathcal{C}$.
 - $\mathcal{C}$ runs $(x_1, \dots, x_n; y) \leftarrow D(g, x)$, $x \leftarrow_{\$} \mathbb{Z}_q$, and sends $y, \{x_i\}_{i \in J}$ to $\mathcal{A}$.
- *Impersonation:* $\mathcal{A}$ outputs a proof of the form $\sigma = \{(u_i, s_i)\}_{i \in Q}$.

$\mathcal{A}$ wins if σ is valid against y .

Note that successful impersonation is equivalent to condition (3.21) for σ . Security against impersonation attacks should mean that every polynomial-time adversary $\mathcal{A}$ wins the above game with at most negligible advantage. The core requirement behind is that if $\mathcal{A}$ outputs a valid proof $\{(u_i, s_i)\}_{i \in Q}$ then it should necessarily know x_j for some $j \notin J$; i.e., impersonation should be infeasible without knowledge of at least t secret shares. We will refer to this design requirement as the *OSST knowledge property*.

This is instance of a more general pattern. Security notions for Schnorr-based protocols are intermediated by some knowledge property like above, which is the pragmatic design requirement behind. Note that the above knowledge property cannot be formally reduced to ordinary Schnorr in a straight-line fashion because the provers commit to unrelated group elements. Consequently, it must be derived by means of fork and extraction in the random oracle model; more accurately, an appropriate extraction procedure should itself be reproducible within any forkable impersonation attack against the interactive variant of the protocol (cf. Section 3.1.1). In this case, assuming that $\mathcal{A}$ wins with some non-negligible advantage, one has to construct a DL-adversary $\mathcal{A}^*$ who perfectly simulates $\mathcal{C}$ while also having forkable black-box access to $\mathcal{A}$; upon successful completion of $\mathcal{A}$'s first attack, $\mathcal{A}^*$ should be able to fork $\mathcal{A}$ at some special point, let it complete a second successful attack with non-negligible advantage and finally solve DL by extracting the desired logarithm from the two partially differing transcripts.

In many distributed protocols, this degree of generality is bounded by certain conditions that are required for achieving extractability, usually reflecting the fact that no inherent relations are known among the logarithms of the commitments. In the extreme, the provers may even commit to unrelated group elements, as is the case with OSST. In this specific case, the only currently available extraction formula is equation (3.14) from the key-recovery attack of Section 3.4.3, which is insufficient for retrieving x_j unless $\{r_i\}_{i \in J}$ can be known along with $\{x_i\}_{i \in J}$. In view of Game 4.1, this means that OSST security cannot be reduced to DL alone. We need further assumptions in order for security to become provable or, what is equivalent, reduce it to a stronger hardness assumption than DL. Below we proceed to gradually restricting the general attack model for the sake of provability.

4.1.2 Extractability and $(t - 1)$ -OMDL hardness

Recall the circumstances under which the attack of Section 3.4.3 becomes possible. First, in order to be able to derive equation (3.14), $\mathcal{A}^*$ must control the verifier and some shareholders $J \subset \{1, \dots, n\}$ with $|J| = t - 1$; in the above reduction schema, this condition corresponds to the fact that $\mathcal{A}^*$ simulates $\mathcal{C}$ while also controlling $\mathcal{A}$ and consequently the corrupted shareholders, with its control over the latter involving at least knowledge of $\{x_i\}_{i \in J}$. Second, in order to be able to effectively compute x_j , $\mathcal{A}^*$ must know the logarithms $r_i = \log u_i$, $i \in J$. However, the black-box adversary of Game 4.1 is not generally assumed to know them, meaning that the reduction algorithm $\mathcal{A}^*$ cannot be assumed to learn them by simply accessing $\mathcal{A}$'s memory – unless the latter fully controlled the corrupted shareholders' devices or could at least subvert their random number generator or other security module. We can codify such scenarios under the assumption that $\mathcal{A}^*$ is allowed to issue at most $t - 1$ queries to a hypothetical DL oracle; extraction of x_j would then become possible, suggesting that security might as well be reducible to the $(t - 1)$ -OMDL assumption (cf. Appendix C). Note that $(t - 1)$ -OMDL, $t > 1$, is indeed a stronger assumption than DL, plausibly yielding the missing conditions of extractability.

Using $\{u_i\}_{i \in J}$ in the above fashion involves two assumptions. First, deriving equation (3.14) requires that these elements be made explicitly available to $\mathcal{A}^*$ before the forking moment, so that they can be kept fixed for $\mathcal{A}$'s forked impersonation attempt. Second, $\{u_i\}_{i \in J}$ should be uniform from $\mathcal{A}^*$'s viewpoint, so that the rules of Game C.1 are not violated. Both requirements are met if we assume that $\mathcal{A}$ fabricates $\{(u_i, s_i)\}_{i \in Q}$ for uniformly sampled commitments $u_i \leftarrow_{\$} \mathbb{Z}_q$, $i \in J$. It should be stressed that this is a non-trivial premise that genuinely restricts the attack model by excluding the case of internally structured fabricated commitments. Nevertheless, it is probably not a far-fetched assumption; since H is a random oracle, the very last challenge injects perfect entropy to both sides of (3.30), so that the particular choice of u_i , $i \in J$, has good chances to be indifferent regarding $\mathcal{A}$'s strategy for fabricating u_j and s_i , $i \in Q$. We conclude with the following tradeoff between Game 4.1 and security provability by means of fork and extraction.

Game 4.2. Given adversary $\mathcal{A}$ and challenger $\mathcal{C}$,

- *Attack statement and corruption:*
 - $\mathcal{A}$ chooses $J \subset \{1, \dots, n\}$ with $|J| = t - 1$ and sends it along with some $Q \subset \{1, \dots, n\}$ to its challenger $\mathcal{C}$.
 - $\mathcal{A}$ chooses $Q \subset \{1, \dots, n\}$ with $Q \subset J$ or $J \subset Q$ and sends it along with J to its challenger $\mathcal{C}$.
 - $\mathcal{C}$ runs $(x_1, \dots, x_n; y) \leftarrow D(g, x)$, $x \leftarrow_{\$} \mathbb{Z}_q$, and sends $y, \{x_i\}_{i \in J}$ to $\mathcal{A}$.
- *Interception:* $\mathcal{C}$ samples $u_i \leftarrow_{\$} \mathbb{G}$, $i \in J$ and sends $\{u_i\}_{i \in J}$ to $\mathcal{A}$.
- *Impersonation:* $\mathcal{A}$ outputs a proof of the form $\{(u_i, s_i)\}_{i \in Q}$.

$\mathcal{A}$ wins if $\{(u_i, s_i)\}_{i \in Q}$ verifies against y .

4.1.3 Implications of $(t - 1)$ -OMDL hardness

Observe that no restrictions have yet been imposed on $|Q|$, allowing $\mathcal{A}$ to output a valid proof with less than t components. This is plausible because proofs of this kind exist in abundance (cf. Remarks 3.25 and 3.28). However, under $(t - 1)$ -OMDL hardness, fabricating such proofs can be ruled out as infeasible with a straight-line reduction. Let $\mathcal{A}$ be an adversary that

wins Game 4.2 with $Q \subset J$ (i.e., $|Q| < t$). Write $J = \{i_1, \dots, i_{t-1}\}$ with $i_1 < \dots < i_{t-1}$, so that $Q = \{i_1, \dots, i_m\}$ for some $m \in \{1, \dots, t-1\}$. When given $w_0, \dots, w_{t-1}$ by its challenger (cf. Game C.1), $\mathcal{A}^*$ sets $y \leftarrow w_0$ and $u_{i_k} \leftarrow w_k$ for $1 \leq k \leq t-1$. Next, $\mathcal{A}^*$ samples $x_i \leftarrow_{\$} \mathbb{Z}_q$, $i \in J$, and forwards $y, \{x_i\}_{i \in J}$ to $\mathcal{A}$. By the security property of Shamir's sharing (cf. Remark B.7), y and $\{x_i\}_{i \in J}$ uniquely determine a (n, t) -sharing $x_1, \dots, x_n$ of some $x \equiv \log y$ with $\mathcal{A}^*$ perfectly simulating $\mathcal{C}$. Next, $\mathcal{A}^*$ sends $\{u_i\}_{i \in J}$ to $\mathcal{A}$ and lets it proceed to impersonation; since $\mathcal{C}$ has been perfectly simulated, $\mathcal{A}$ outputs a valid proof $\{(u_i, s_i)\}_{i \in Q}$ with probability equal to its advantage. Condition (3.21) is then satisfied and $\mathcal{A}^*$ can recover x by issuing at most $t-1$ DL-queries. More accurately, $\mathcal{A}^*$ queries

$$r_{i_k} \leftarrow \mathcal{O}_{\mathbb{G}}^{\text{dlog}}(u_{i_k}), \quad 1 \leq k \leq t-1,$$

and uses r_{i_k} , $1 \leq k \leq m$, to compute x . Next, $\mathcal{A}^*$ sets $z_0 \leftarrow x$ and $z_k \leftarrow r_{i_k}$, $1 \leq k \leq t-1$, and wins by forwarding $z_0, \dots, z_{t-1}$ to its challenger.

In short, if $\mathcal{A}$ wins, then $\mathcal{A}^*$ solves $(t-1)$ -OMDL with equal time complexity and advantage. This reduction allows us to restrict attention to the case $J \subset Q$ (i.e. $|Q| \geq t$), excluding the possibility of impersonating less than t shareholders. This is evidently a security requirement (cf. Remark 3.28), trivially captured by the $(t-1)$ -OMDL assumption.

4.2 Security against impersonation attacks – working model

Applying the conclusion of Section 4.1.3 to Game 4.2, we conclude with the following working model for impersonation attacks.

Game 4.3. ($\text{OSST}_{\mathbb{G}, n, t}^{\text{imp}} - \text{Impersonation}$) Given adversary $\mathcal{A}$ and challenger $\mathcal{C}$,

- *Attack statement and corruption:*
 - $\mathcal{A}$ chooses $J \subset \{1, \dots, n\}$ with $|J| = t-1$ and sends it along with some $j \in \{1, \dots, n\} \setminus J$ to the challenger. Denote $Q = J \cup \{j\}$.
 - $\mathcal{C}$ runs $(x_1, \dots, x_n; y) \leftarrow D(g, x)$, $x \leftarrow_{\$} \mathbb{Z}_q$, and sends $y, \{x_i\}_{i \in J}$ to $\mathcal{A}$.
- *Interception:* $\mathcal{C}$ samples $u_i \leftarrow_{\$} \mathbb{G}$, $i \in J$, and sends $\{u_i\}_{i \in J}$ to $\mathcal{A}$.
- *Impersonation:* $\mathcal{A}$ outputs a proof of the form $\sigma = \{(u_i, s_i)\}_{i \in Q}$.

$\mathcal{A}$ wins if $\mathcal{V}(y, \sigma) = \text{true}$.

The probability that $\mathcal{A}$ wins Game 4.3 is denoted by

$$\text{Adv}_{\mathbb{G}, n, t}^{\text{OSST}}[\mathcal{A}]. \quad (4.4)$$

OSST security against impersonation attacks (over $\mathbb{G}$) should mean that (4.4) is negligible for every choice of n, t and every polynomial-time adversary $\mathcal{A}$.

Remark 4.5. (*Adversarial components*) By the security property of Shamir's secret sharing (cf. Remark B.7), j and J (and consequently Q) may be regarded as fixed for $\mathcal{A}$. In particular, $\mathcal{A}$ is a pair of non-interactive algorithms $(\mathcal{A}_0, \bar{\mathcal{A}})$ of the form

$$(j, J) \leftarrow \mathcal{A}_0(\cdot), \quad \{(u_i, s_i)\}_{i \in Q} \leftarrow \bar{\mathcal{A}}(y, \{x_i\}_{i \in J}, \{u_i\}_{i \in J}),$$

where $\bar{\mathcal{A}}$ consumes $\mathcal{A}$'s view during the impersonation phase in order to fabricate u_j and $\{s_i\}_{i \in Q}$ such that $\{(u_i, s_i)\}_{i \in Q}$ is a valid proof. Note that, when using $\mathcal{A}$ in a black-box reduction, the logarithm of the fabricated commitment u_j cannot be assumed to be known and we will normally have to eliminate it from our computations.

Remark 4.6. (*Deterministic adversaries*) For every adversary $\mathcal{A}$ attacking Game 4.3, there exists a deterministic adversary $\mathcal{A}'$ of equal time complexity such that

$$\text{Adv}_{\mathbb{G}, n, t}^{\text{OSST}}[\mathcal{A}'] \geq \text{Adv}_{\mathbb{G}, n, t}^{\text{OSST}}[\mathcal{A}].$$

This follows from a generic argument about any adversary $\mathcal{A}$ attacking a sufficiently simple game $\mathbb{G}$. Consider the residual deterministic strategies $\mathcal{A}_1, \dots, \mathcal{A}_m$ obtained by fixing the possible values of $\mathcal{A}$'s random coins and let p_i denote the probability that $\mathcal{A}$ runs $\mathcal{A}_i$ (which is itself an adversary for $\mathbb{G}$). We then have

$$\text{Adv}^{\mathbb{G}}[\mathcal{A}] = \sum_{i=1}^m p_i \text{Adv}^{\mathbb{G}}[\mathcal{A}_i].$$

Since $p_1 + \dots + p_m = 1$, we conclude that $\text{Adv}^{\mathbb{G}}[\mathcal{A}_i] \geq \text{Adv}^{\mathbb{G}}[\mathcal{A}]$ for some $i \in \{1, \dots, m\}$.

5 Security proof

In this section, we prove OSST security against impersonation attacks (cf. Game 4.3) under OMDL hardness (cf. Appendix C) in the random oracle model. The rationale behind the attack model is developed throughout Section 4.1. Our reduction generalizes the rewinding argument of the non-distributed case in order to extract the unknown x_j . This is a non-trivial generalization. By the key-recovery attack of Section 3.4.3, the extractor carries a residue from the $t - 1$ degrees of freedom in perturbing a valid proof with fixed commitments (cf. Remark 3.25); it becomes useful only if the logarithms of $\{u_i\}_{i \in J}$ are known to $\mathcal{A}^*$, which is the reason for reducing to the $(t - 1)$ -OMDL hardness assumption. Moreover, extraction requires fixing all but one challenges, making rewinding useless in the threshold setting. We overcome this obstacle by using the Local Forking Lemma (cf. Appendix E) and ensuring that the conditions for its applicability are fulfilled. The attack simulator is presented in Section 5.3 and its components are defined in Sections 5.1 and 5.2.

5.1 Index mapping formalism

Given $J \subset \{1, \dots, n\}$, $|J| = m$, we can represent any collection $\alpha_1, \dots, \alpha_m$ in the form $\{\beta_i\}_{i \in J}$ by using the **Zip** operator, Algorithm 1. Clearly, **Zip** preserves uniform sampling.

Lemma 1. *Given $J \subset \{1, \dots, n\}$, $|J| = m \geq 1$, the probability distribution*

$$\{\text{Zip}(\alpha_1, \dots, \alpha_m; J) : \alpha_i \leftarrow_{\S} A, i = 1, \dots, m\}$$

is identical to the distribution $\{\{\beta_i\}_{i \in J} : \beta_i \leftarrow_{\S} A, i \in J\}$.

Proof. Follows directly from the definition of **Zip**, Algorithm 1. □

Algorithm 1 $\text{Zip}(\alpha_1, \dots, \alpha_m; J)$

$\{i_1, \dots, i_m\} \leftarrow J$ with $i_1 < \dots < i_m$
for $k = 1, \dots, m$
 $\beta_{i_k} \leftarrow \alpha_k$
return $\{\beta_i\}_{i \in J}$

Algorithm 2 $\text{Zip}^{-1}(\{\beta_i\}_{i \in J})$

$\{i_1, \dots, i_m\} \leftarrow J$ with $i_1 < \dots < i_m$
for $k = 1, \dots, m$
 $\alpha_k \leftarrow \beta_{i_k}$
return $\alpha_1, \dots, \alpha_m$

5.2 Shamir's sharing simulation

Let $y^* \in \mathbb{G}$ and $x_1^*, \dots, x_{t-1}^* \in \mathbb{Z}_q$ with $t \leq n$. Given $J \subset \{1, \dots, n\}$ with $|J| = t - 1$ and $j \in \{1, \dots, n\} \setminus J$, we define

$$(\{x_i\}_{i \in J}, y) \leftarrow \mathcal{D}(y^*, x_1^*, \dots, x_{t-1}^*; j, J) \quad (5.1)$$

to be Algorithm 3. Its properties are summarized as follows.

Lemma 5.2. *Let $J \subset \{1, \dots, n\}$, $|J| = t - 1$ with $t \leq n$, and $j \in \{1, \dots, n\} \setminus J$. Given (5.1), there exists a unique (n, t) -sharing $(x_1, \dots, x_n; y)$ such that*

$$\{x_i\}_{i \in J} = \text{Zip}(x_1^*, \dots, x_{t-1}^*) \wedge y^* = g^{x_j}. \quad (5.3)$$

Moreover, the probability distribution

$$\{(\{x_i\}_{i \in J}, y) : \left\{ \begin{array}{l} (\{x_i\}_{i \in J}; y) \leftarrow \mathcal{D}(y^*, x_1^*, \dots, x_{t-1}^*; j, J) \\ y^* \leftarrow_{\$} \mathbb{G}, \quad x_i^* \leftarrow_{\$} \mathbb{Z}_q, \quad 1 \leq i \leq t-1 \end{array} \right\}\}$$

is identical to $\{(\{x_i\}_{i \in J}, y) : (x_1, \dots, x_n; y) \leftarrow D(g, x), x \leftarrow_{\$} \mathbb{Z}_q\}$.

Proof. By interpolation, $x_1^*, \dots, x_{t-1}^*$ and y^* uniquely determine a sharing subject to (5.3), with the possible outputs of $\mathcal{D}$ mapping to those of D bijectively. The equality of distributions follows from the security property of Shamir's sharing (cf. Remark B.7). $\square$

Algorithm 3 $\mathcal{D}(y^*, x_1^*, \dots, x_{t-1}^*; j, J)$

1: Set $y_j \leftarrow y^*$
2: $\{x_i\}_{i \in J} \leftarrow \text{Zip}(x_1^*, \dots, x_{t-1}^*)$
3: Set $y_i \leftarrow g^{x_i}$, $i \in J$
4: $Q \leftarrow J \cup \{j\}$
5: $y \leftarrow \prod_{i \in Q} y_i^{\lambda_i}$
6: **return** $\{x_i\}_{i \in J}, y$

5.3 Simulated impersonation attack

We define an algorithm $\mathcal{S}[\mathcal{A}]$ that simulates Game 4.3 for an embedded adversary $\mathcal{A}$, provided that H is modelled as a random oracle. Roughly speaking, $\mathcal{S}[\mathcal{A}]$ accepts the ingredients of $\mathcal{A}$'s view and outputs a proof along with a relevant success index. The simulation is perfect in the sense that, if the algorithm is fed with uniform input, the probability of success equals the adversarial advantage of winning the real attack game.

More accurately, given a black-box adversary $\mathcal{A}$ to game $\text{OSST}_{\mathbb{G},n,t}^{\text{imp}}$, the desired simulation is a wrapper of $\mathcal{A}$ of the form

$$(k, (j, \sigma)) \leftarrow \mathcal{S}[\mathcal{A}](y^*, x_1^*, \dots, x_{t-1}^*, w_1, \dots, w_{t-1}; h_1, \dots, h_m),$$

arranging the input values appropriately in order to emulate $\mathcal{A}$'s execution environment. The values $y^*, x_1^*, \dots, x_{t-1}^*$ determine the attacked (n, t) -sharing, including the shares $\{x_i\}_{i \in J}$ that are known to $\mathcal{A}$. Similarly, $w_1, \dots, w_{t-1}$ yield the intercepted commitments $\{u_i\}_{i \in J}$. Next, the embedded replica of $\mathcal{A}$ consumes its view in order to fabricate a purportedly valid proof σ with the difference that, instead of invoking H directly, it queries an embedded hash oracle $\mathcal{O}^{\text{hash}}$ with tape $h_1, \dots, h_m$ (cf. Appendix D). If needed, $\mathcal{A}$ may be reprogrammed without loss of generality to query hashes for all the commitments it sees (including the one fabricated by itself), so that $m \geq 1$. Finally, an OSST-verifier $\mathcal{V}$ is included with access to the same hash oracle to verify $\mathcal{A}$'s output. The exact procedure is shown in Algorithm 4.

Algorithm 4 $\mathcal{S}[\mathcal{A}](y^*, x_1^*, \dots, x_{t-1}^*, w_1, \dots, w_{t-1}; h_1, \dots, h_m)$

```

1:  $h \leftarrow [h_1, \dots, h_m]$ ,  $\nu \leftarrow 1$ ,  $\text{Map}_1 = \emptyset$ ,  $\text{Map}_2 = \emptyset$  // Setup  $\mathcal{O}^{\text{hash}}$ 
2:  $(j, J) \leftarrow \mathcal{A}_0(\cdot)$  // Attack statement
3:  $(\{x_i\}_{i \in J}, y) \leftarrow \mathcal{D}(y^*, x_1^*, \dots, x_{t-1}^*; j, J)$  // Corruption
4:  $\{u_i\}_{i \in J} \leftarrow \text{Zip}(w_1, \dots, w_{t-1})$  // Interception
5:  $\{(u_i, s_i)\}_{i \in Q} \leftarrow \bar{\mathcal{A}}(y, \{x_i\}_{i \in J}, \{u_i\}_{i \in J})$  // Impersonation
6:  $res \leftarrow \mathcal{V}(y, \{(u_i, s_i)\}_{i \in Q})$ 
7: for  $i \in Q$ 
8:    $c_i \leftarrow \text{Map}_1[u_i]$ 
9:  $\sigma \leftarrow \{(u_i, c_i, s_i)\}_{i \in Q}$ 
10:  $k \leftarrow \text{Map}_2[u_j]$ 
11: if  $res = \text{true}$ 
12:   return  $(k, (j, \sigma))$ 
13: return  $(0, (j, \sigma))$ 

```

The hash oracle $\mathcal{O}^{\text{hash}}$ is initialized with tape $[h_1, \dots, h_m]$ and initially empty lookup tables. After $\mathcal{A}$ makes its attack statement, the virtual challenger simulates the unique sharing $(x_1, \dots, x_n; y) \in D_x$, $x \in \mathbb{Z}_q$, that is subject to (5.3); in particular, $x_j = \log y^*$. It then generates commitments $\{u_i\}_{i \in J}$ and feeds them along with y and $\{x_i\}_{i \in J}$ to $\mathcal{A}$, who proceeds to impersonation and generates a proof of the expected form, i.e., such that $Q = J \cup \{j\}$.

Observe that the random oracle queries are implicitly issued at step 5, including the challenges $c_i \leftarrow \mathcal{O}^{\text{hash}}(u_i)$, $i \in Q$, which are collected and attached to the fabricated proof for later usage (steps 7-9). We also locate the unique index $k \in \{1, \dots, m\}$ for which $c_j = h[k]$ (step 10), indicating that the fabricated commitment u_j was submitted at the k -th distinct query. If $\mathcal{A}$ succeeds in its impersonation attempt, the simulation returns $(k, (j, \sigma))$; otherwise, it returns $(0, (j, \sigma))$ to indicate failure.

Proposition 5.4. (OSST $_{\mathbb{G},n,t}^{\text{imp}}$ – Perfect simulation) *If H is modelled as a random oracle, then $\mathcal{S}[\mathcal{A}]$ simulates Game 4.3 perfectly if invoked with uniformly random input, in the sense that the real and simulated game executions are indistinguishable from the adversary's viewpoint. More accurately, the probability distribution of $\mathcal{A}$'s view in the real attack game is identical to the distribution of $\bar{\mathcal{A}}$'s input in the invocation*

$$\begin{aligned} (k, (j, \sigma)) &\leftarrow \mathcal{S}[\mathcal{A}](y^*, x_1^*, \dots, x_{t-1}^*, w_1, \dots, w_{t-1}; h_1, \dots, h_m), \\ y^* &\leftarrow_{\$} \mathbb{G}, \quad x_i^* \leftarrow_{\$} \mathbb{Z}_q, \quad w_i \leftarrow_{\$} \mathbb{G}, \quad 1 \leq i \leq t-1, \\ h_\nu &\leftarrow_{\$} \mathbb{Z}_q, \quad 1 \leq \nu \leq m \end{aligned} \tag{5.5}$$

In the context of this simulation, if $\bar{\mathcal{A}}$ succeeds in its impersonation attempt, i.e., $k \geq 1$, the input value h_k maps to $c_j \equiv H(u_j)$ of the real game execution.

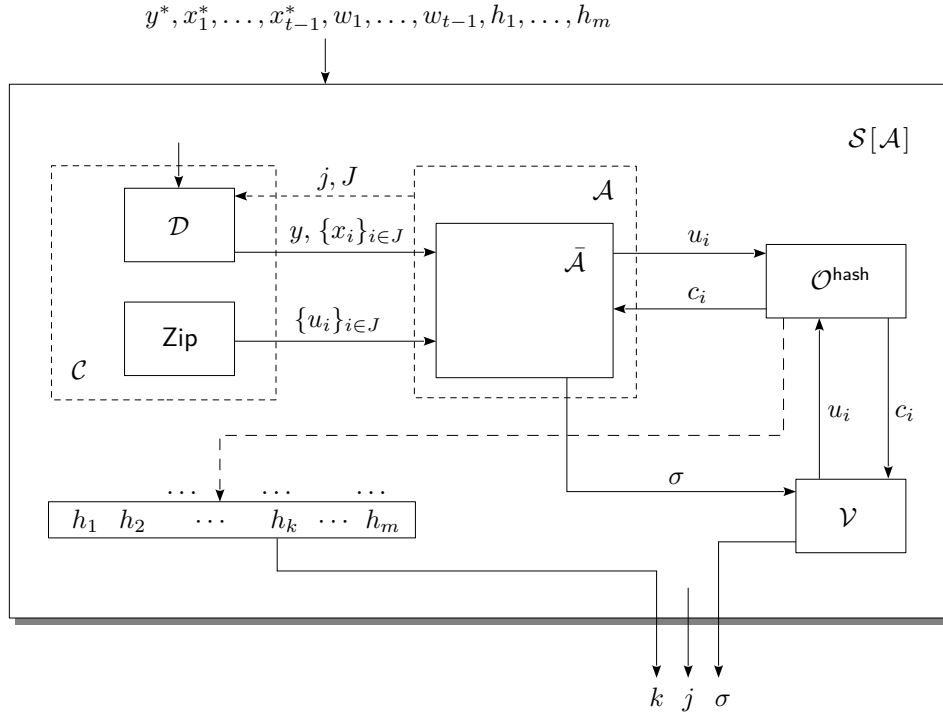


Figure 1: The OSST $_{\mathbb{G},n,t}^{\text{imp}}$ simulator

Proof. In the real game execution, the adversarial view consists of the compromised secret shares $\{x_i\}_{i \in J}$ along with the combined public key y , the intercepted commitments $\{u_i\}_{i \in Q}$ and the hashes queried during the impersonation phase. These three blocks are independent from each other; clearly, the first and second blocks are independent, while the third block is independent from the others because H is a random oracle. It thus suffices to check that the respective blocks have identical distributions in the simulated execution (5.5). The first block is perfectly simulated due to Lemma 5.2. The second block is also perfectly simulated due to Lemma 1. Finally, the third block is perfectly simulated due to Lemma D.1. The fact that $h_k = h[k]$ corresponds to $H(u_j)$ follows directly from step 10, Algorithm 4. $\square$

We denote by $\text{acc}(\mathcal{S}[\mathcal{A}])$ the probability that the embedded adversary $\bar{\mathcal{A}}$ succeeds in the context of a simulated impersonation attack when run with uniformly random input. This correlates with the real adversarial advantage as follows.

Corollary 5.6. *Given an adversary $\mathcal{A}$ for Game 4.3, there holds*

$$\text{acc}(\mathcal{S}[\mathcal{A}]) = \text{Adv}_{\mathcal{G},n,t}^{\text{OSST}}[\mathcal{A}] \quad (5.7)$$

Proof. Direct consequence of Proposition 5.4. $\square$

5.4 Local fork and extraction

In the non-distributed case $n = 1$, the simulator $\mathcal{S}[\mathcal{A}]$ yields the execution environment of the rewindable adversary $\mathcal{A}$ that is usually employed to reduce the security of ordinary Schnorr identification to the DL hardness assumption in the random oracle model. In the distributed case $t \geq 1$, the extractor is generalized as (3.14), provided that the forked attack runs with the same hashes except for $c_j \leftarrow H(u_j)$. This implies that the rewinding argument cannot be effective in the threshold setting; indeed, resetting the attack exactly before $\mathcal{A}$ sends u_j does not guarantee that subsequent oracle responses remain the same and, in fact, they differ almost certainly. However, when invoking $\mathcal{S}[\mathcal{A}]$, we predefine the hashes $h_1, \dots, h_m$ returned by the embedded oracle in respective order, which allows to repeat the simulation with the same hashes except for $c_j \equiv h_k$. In this case, the Local Forking Lemma (cf. Appendix E) becomes applicable in place of rewinding, provided that $\mathcal{A}$ is deterministic (cf. Remark 4.6). Recall that the local fork of $\mathcal{S}[\mathcal{A}]$ is an algorithm of the form

$$(b, \text{out}) \leftarrow \text{LocalFork}_{\mathcal{S}[\mathcal{A}]}(y^*, x_1^*, \dots, x_{t-1}^*, w_1, \dots, w_{t-1})$$

(cf. Appendix E) operating as follows. If the embedded adversary $\mathcal{A}$ succeeds in its first impersonation attempt, the fork repeats the simulation with the same input values except for the k -th predefined hash, which is afresh uniformly sampled as h_k^* ; otherwise, the fork aborts. When the second simulation completes, the fork checks if $\mathcal{A}$ succeeds again under the condition $h_k^* \neq h_k$; in this case, it returns the consecutive outputs $\tau \equiv (j, \sigma)$ and $\tau^* \equiv (j, \sigma^*)$ of both executions; otherwise, it aborts. Evidently, $\text{forkAcc}(\mathcal{S}[\mathcal{A}])$ expresses here the probability that $\mathcal{A}$ succeeds both in its first and second impersonation attempt when consuming the same uniformly random input except for the k -th distinct hash. The consecutive outputs are correlated as follows.

Proposition 5.8. (*Local fork and extraction*) *Let $\mathcal{A}$ be a deterministic adversary for Game 4.3 and $\tau = (j, \sigma)$, $\tau^* = (j, \sigma^*)$ be outputs of a non-aborting execution*

$$(1, (\tau, \tau^*)) \leftarrow \text{LocalFork}_{\mathcal{S}[\mathcal{A}]}(y^*, x_1^*, \dots, x_{t-1}^*, w_1, \dots, w_{t-1})$$

Then σ, σ^* are of the form $\{(u_i, c_i, s_i)\}_{i \in Q}, \{(u_i, c_i^*, s_i^*)\}_{i \in Q}$ and subject to the constraint

$$c_j \neq c_j^* \wedge c_i = c_i^*, \forall i \in Q \setminus \{j\}. \quad (5.9)$$

Moreover, the extraction formula

$$x_j = \frac{s_j - s_j^*}{c_j - c_j^*} + \frac{1}{\mu_j} \sum_{i \in J} \left(\mu_i(s_i - r_i - c_i x_i) - \mu_i^*(s_i^* - r_i - c_i x_i) \right) \quad (5.10)$$

holds true for $x_j \equiv \log y^*$, where $r_i \equiv \log u_i$ and $\{\mu_i\}_{i \in Q}, \{\mu_i^*\}_{i \in Q}$ are the weights of σ and σ^* respectively.

Proof. Let $\sigma = \{(u_i, c_i, s_i)\}_{i \in Q}$, $\sigma^* = \{(u_i^*, c_i^*, s_i^*)\}_{i \in Q}$, so that $u_i^* = u_i, \forall i \neq j$. Fabricating u_j completes before submitting it to the hash oracle and is thus unaffected by the k -th distinct hash. Since $\mathcal{A}$ is deterministic and its view is the same in both simulations except for the latter hash, we conclude that $u_j = u_j^*$. Constraint (5.9) follows because $c_j^* = h_k^* \neq h_k = c_j$ and the rest challenges are drawn from $\{h_\nu\}_{\nu \neq k}$. We now proceed to extraction. Since both σ, σ^* are valid, Corollary 3.29 implies

$$\sum_{i \in Q} \mu_i(s_i - r_i - c_i x_i) = 0, \quad \sum_{i \in Q} \mu_i^*(s_i^* - r_i - c_i^* x_i) = 0. \quad (5.11)$$

Since $c_i = c_i^*$ for $i \neq j$, we get $\mu_j = \mu_j^*$ (cf. Definition 3.7). Subtracting terms in (5.11) and applying this remark yields

$$\mu_j(s_j - s_j^* - (c_j - c_j^*)x_j) + \sum_{i \in J} (\mu_i(s_i - r_i - c_i x_i) - \mu_i^*(s_i^* - r_i - c_i x_i)) = 0.$$

Extraction follows because $c_j - c_j^* \neq 0$. $\square$

5.5 Main theorem

Theorem 5.12. (OSST – Security against impersonation attacks) *Given $1 \leq t \leq n < q$, OSST $_{\mathbb{G}, n, t}$ (cf. Protocol 3.1) is secure against impersonation attacks under $(t-1)$ -OMDL hardness in the random oracle model. In particular, if H is modelled as a random oracle, for every polynomial-time adversary $\mathcal{A}$ that attacks Game 4.3 there exists an equal time complexity adversary $\mathcal{A}^*$ that attacks Game C.1 with*

$$\text{Adv}_{\mathbb{G}, t-1}^{\text{OMDL}}[\mathcal{A}^*] \geq \frac{1}{q_{\text{ro}} + 1} \text{Adv}_{\mathbb{G}, n, t}^{\text{OSST}}[\mathcal{A}]^2, \quad (5.13)$$

where $q_{\text{ro}} + 1$ is the number of distinct queries issued by $\mathcal{A}$ to the random oracle.

Proof. We construct $\mathcal{A}^*$ as a wrapper of $\text{LocalFork}_{\mathcal{S}[\mathcal{A}]}$; after forking a successful impersonation attack, the wrapper should be able to recover the unknown share by issuing at most $t-1$ discrete-logarithm queries and then use it to solve $\text{OMDL}_{\mathbb{G}, t-1}$ with comparable advantage.

We may assume without loss of generality that $\mathcal{A}$ is deterministic (cf. Remark 4.6). When given $w_0, \dots, w_{t-1}$ by its challenger (cf. Game C.1), $\mathcal{A}^*$ operates as shown in Algorithm 5.

Algorithm 5 $\mathcal{A}^*(w_0, w_1, \dots, w_{t-1})$

- 1: Set $y^* \leftarrow w_0$
- 2: Set $x_i^* \leftarrow_{\$} \mathbb{Z}_q$, $1 \leq i \leq t-1$
- 3: $(b, \text{out}) \leftarrow \text{LocalFork}_{\mathcal{S}[\mathcal{A}]}(y^*, x_1^*, \dots, x_{t-1}^*, w_1, \dots, w_{t-1})$
- 4: **if** $b = 0$
- 5: **return** $\perp$
- 6: Parse $(\tau, \tau^*) \leftarrow \text{out}$
- 7: Parse $(j, \{(u_i, c_i, s_i)\}_{i \in Q}) \leftarrow \tau$
- 8: Parse $(j, \{(u_i, c_i^*, s_i^*)\}_{i \in Q}) \leftarrow \tau^*$
- 9: Set $J \leftarrow Q \setminus \{j\}$
- 10: Compute the Lagrange interpolation coefficients $\{\lambda_i\}_{i \in Q}$
- 11: Compute

$$\mu_j \leftarrow \lambda_j \prod_{i \in J} c_i$$

- 12: **for** $i \in J$, compute

$$\mu_i \leftarrow \lambda_i \prod_{k \in Q \setminus \{i\}} c_k, \quad \mu_i^* \leftarrow \lambda_i \prod_{k \in Q \setminus \{i\}} c_k^*$$

- 13: Set $\{x_i\}_{i \in J} \leftarrow \text{Zip}(x_1^*, \dots, x_{t-1}^*; J)$
- 14: Query $r_i \leftarrow \mathcal{O}_{\mathbb{G}}^{\text{dlog}}(u_i)$, $i \in J$ // $t-1$ queries
- 15: Compute

$$x_j \leftarrow \frac{s_j - s_j^*}{c_j - c_j^*} + \frac{1}{\mu_j} \sum_{i \in J} \left(\mu_i (s_i - r_i - c_i x_i) - \mu_i^* (s_i^* - r_i - c_i x_i) \right)$$

- 16: Set $z_0 \leftarrow x_j$
 - 17: Set $(z_1, \dots, z_{t-1}) \leftarrow \text{Zip}^{-1}(\{r_i\}_{i \in J})$
 - 18: **return** $z_0, z_1, \dots, z_{t-1}$
-

Observe that $\mathcal{A}^*$ aborts if the fork aborts, i.e., unless both attempts of $\mathcal{A}$ are successful; otherwise, the fork fulfills the conditions of Proposition 5.8 and the proofs fabricated by $\mathcal{A}$ are as shown at steps 7 and 8. By definition of $\mathcal{S}[\mathcal{A}]$ (cf. Algorithm 4), the attacks of step 3 unfold against the sharing $(x_1, \dots, x_n; y)$ that is determined by

$$\{x_i\}_{i \in J} = \text{Zip}(x_1^*, \dots, x_{t-1}^*) \wedge y^* = g^{x_j} \quad (5.14)$$

(cf. Lemma 5.2). Similarly, $\{u_i\}_{i \in J}$ comprise of the input values $w_1, \dots, w_{t-1}$; in particular,

$$\{u_i\}_{i \in J} = \text{Zip}(w_1, \dots, w_{t-1}) \quad (5.15)$$

(cf. Algorithm 4, step 4). In steps 9-14, $\mathcal{A}^*$ computes the parameters needed for step 15. The shares $\{x_i\}_{i \in J}$ are retrieved using the first part of (5.14) (step 13) while the logarithms of u_i , $i \in J$, are retrieved by submitting them to the DL oracle (step 14). Since $y^* = w_0$, Proposition 5.8 ensures that step 15 yields $z_0 = \log w_0$. Similarly, due to (5.15), step 17 yields $z_i = \log w_i$, $1 \leq i \leq t-1$. In short, $z_0, z_1, \dots, z_{t-1}$ is the solution to the original problem instance and $\mathcal{A}^*$ has issued at most $t-1$ distinct queries to the DL oracle.

This discussion shows that $\mathcal{A}^*$ solves $\text{OMDL}_{\mathbb{G}, t-1}$ exactly if step 3 is a non-aborting execution. Since $w_0, w_1, \dots, w_{t-1}$ are uniformly sampled by $\mathcal{A}^*$'s challenger, we conclude

$$\text{Adv}_{\mathbb{G}, t-1}^{\text{OMDL}}[\mathcal{A}^*] = \text{forkAcc}(\mathcal{S}[\mathcal{A}]).$$

Let $m = q_{\text{ro}} + 1$ be the number of distinct queries issued by $\mathcal{A}$ to $\mathcal{O}^{\text{hash}}$ in the context of $\mathcal{S}[\mathcal{A}]$. The desired inequality follows from the Local Forking Lemma and Corollary 5.6. $\square$

Remark 5.16. (*Leaked public shares*) The black-box adversary of Game 4.3 is not assumed to be given the public shares $y_i = g^{x_i}$, $1 \leq i \leq n$, because these are not in general advertised. However, a real-life adversary probably knows them before mounting its actual attack. For instance, a man-in-the-middle who intercepts multiple identification sessions can infer the public keys of the involved shareholders in the form (3.12); or, if the shareholders store $\{y_i\}_{i=1}^n$ for mutual verification purposes, an attacker who compromises the device of a shareholder can learn them immediately if not erased. The result of Theorem 5.12 remains intact because the embedded adversary $\bar{\mathcal{A}}$ can trivially compute $\{y_k\}_{k \notin J}$ in the form

$$y_k \leftarrow \sqrt[k]{y \left(\prod_{i \in J} y_i^{\lambda_i^k} \right)^{-1}}, \quad k \in \{1, \dots, n\} \setminus J,$$

where $\{\lambda_i^k\}_i$ are the interpolation coefficients for $J \cup \{k\}$ (cf. Remark B.3).

6 Feasibility analysis

We proceed to the feasibility analysis of the OSST protocol through a conservative back-of-the-envelope calculation. We focus on the backend verification process to estimate its execution time in the range 7-40 ms for $|Q| = 100$ on an older x86-64 processor running at 3.5 GHz without cryptographic extensions, dependent on whether multi-exponentiation based optimizations are applied or not. In doing so, we derive a platform agnostic cost model that remains applicable irrespective of hardware constraints, although actual performance is expected to be highly sensitive to the capacity of the CPU cache (if any) for very large $|Q|$'s.

This calculation should be treated as a qualitative explanation within the correct order of magnitude and a suggested optimization roadmap rather than a rigorously validated prediction. Its exact parameters, including both the absolute value of the base unit and the relative costs of the various operations, need to be calibrated through actual profiling on the target hardware. The same holds true for the effect of memory hierarchy, which is particularly relevant for multi-exponentiation and has necessarily been left out of our cost model; nevertheless, we take care to indicate how this effect should be taken into consideration once it has been empirically measured under relatively stable load on the operating system.

Finally, we ignore parallelism, assuming a single-threaded verification process to preserve the modularity and portability of the cost model.

6.1 Elementary costs

We will express execution times in terms of the units defined in Table 1a. We stick to the exponential notation and terminology, meaning that group multiplication, group squaring and exponentiation translate to point addition, point doubling and scalar multiplication respectively in the elliptic curve idiom. While field addition (or subtraction) and hash-into-field are typically ignored, these operations are massively repeated in OSST verification variants, making it safer to include them in our estimations; in particular, the cumulative effect of hashing can be non-negligible over CPUs that do not come with dedicated instruction set extensions (as is still the case with many x86-64 processors).

We will similarly express storage needs in terms of the units defined in Table 1b. Note that the absolute value of $\mathcal{S}$ is always $(1/8) \log q$ bytes while that of $\mathcal{U}$ depends on how group elements are loaded in memory; for example, a P-256 point requires 64 bytes in affine and 94 bytes in Jacobian coordinates.

Table 1: Elementary cost notation

(a) Time cost symbols

Operation	Symbol
Field multiplication	α
Modular inversion	β
Field addition/subtraction	γ
Group multiplication	δ
Group squaring	ϵ
Exponentiation	ζ
Hashing into field	η

(b) Space cost symbols

Element	Symbol
Scalar number in $\mathbb{Z}_q$	$\mathcal{S}$
Group element in $\mathbb{G}$	$\mathcal{U}$

In order to derive concrete time estimates, we will need to express the costs of Table 1a in terms of a common base unit. Since field multiplication and field squaring are the most frequently applied operations (contributing to modular inversion and exponentiation under the hood), it makes sense to employ α as our base unit. It should be stressed that both the absolute value of α and the relative costs of the various operations vary significantly per computing architecture. Before focusing on x86-64 platforms, we provide an overview of these variations; this should help the reader define the scope of our calculations and adapt the suggested optimization strategy to their target hardware constraints.

6.1.1 ECC performance across distinct platforms

Elliptic curves exhibit distinct performance profiles per architecture due to differences in the design of the instruction set. Modern x86-64 and ARM64 are designed for high-throughput but optimize along distinct axes; by contrast, AVR represents the extreme of constrained computing, where even minimal workloads must operate under stringent limits of memory and power consumption.

On x86-64 processors, field multiplication is accelerated through general-purpose rather than dedicated cryptographic extensions. While their respective cost ratios are high, modular inversion and exponentiation are fast in absolute terms because $\alpha \sim 0.1\mu\text{s}$. This is attained through specialized arithmetic logic units (e.g., 256-bit ALUs) and instruction set extensions

(MULX, ADCX/ADOX) that cause the number of required CPU cycles per field multiplication to drop much faster than the algorithmic overhead of its invocation. These optimizations come at the cost of increased power consumption and thermal output, making x86-64 appropriate for a typical server-class verifier, where energy efficiency is a lesser concern and reducing absolute latency remains top priority due to concurrent sessions.

ARM64 processors attain comparable efficiency along with superior performance per Watt through dedicated cryptographic extensions (MADDW, MSUBW, PMULL, PMULL2) that consume significantly less power. The same holds true for extensions like SHA256H and SHA512H that reduce the effect of massive hashing. However, while the absolute value of α is slightly lower, the ratios β/α and ζ/α appear moderately increased as compared to x86-64 processors, suggesting that ARM64 remain mostly appropriate when thermal limits and power budget dominate other constraints. Nevertheless, the gap between x86-64 and ARM64 regarding absolute latency tends to be closing in the recent years.

AVRs operate under ultra low power at a frequency range of 1-20 MHz, with their ALUs handling at most one byte per instruction. They lack hardware acceleration capabilities, with the base unit α having itself a large absolute value. For example, [26] reports 0.726 ms (5355 cycles) per Karatsuba multiplication at 7.3728 MHz; this implies that the cumulative costs of inversion and exponentiation grow prohibitively, although β/α and ζ/α can appear smaller as compared to x86-64. Similarly, algorithmic optimizations based on precomputations are rather limited due to the stringent memory constraints of the microcontroller.

6.1.2 NIST P-256 on x86-64

We pin down representative costs for the elliptic curve group P-256 over modern x86-64 processors. We assume that curve points are represented in Jacobian coordinates (94 bytes per point) that accelerate group operations by reducing the number of required modular inversions.⁸ Note that curve points are usually transmitted or persistently stored in affine coordinates, which incurs a conversion overhead whenever they are loaded in memory for cryptographic purposes. We ignore this overhead by considering it to be I/O in case of bulk or negligible in case of incremental verification (where only one commitment needs to be converted per incoming connection). Regarding the absolute value of α (cf. Table 1a), [1] reports 298 CPU cycles per Montgomery multiplication at 3.5 GHz on an Intel Xeon E3-1247 (2011) processor, i.e., ~ 85.14 ns at the same clock frequency. By rounding for safety, we conclude with Table 2 for working purposes.

Table 2: Fixed base costs for P-256 over x86-64 (3.5 GHz)

Symbol	P-256/x86-64 (3.5 GHz)
α	0.1 μ s
$\mathcal{S}$	32 bytes
$\mathcal{U}$	94 bytes

We now proceed to express the time costs of Table 1a in terms of the base unit α . Note that these ratios are cycle-count invariants per architecture and thus independent of the clock rate or the absolute value of α ; however, they still depend on platform architecture because elliptic curve algorithms build on top of field operations and the relative costs of the

⁸ In the extreme opposite of 8-bit AVR platforms, curve points are usually represented in affine coordinates (cf. [26]), which is aligned with the stringent memory constraints of microcontrollers.

latter differ significantly per instruction set (cf. Section 6.1.1). On x86-64, field addition can be safely ignored while field squaring is typically assumed to be the equivalent of ~ 0.8 field multiplications according to community knowledge and reproducible benchmarks. Below we justify the fixed values of Table 3.

Assuming that modular inversion is based on the Extended Euclidean Algorithm, [18] suggests a cost of ~ 80 field multiplications.⁹ For group multiplication (i.e., point addition in Jacobian coordinates), Table 3.3 from [18] suggests a cost of 12 field multiplications and 4 field squarings. Regarding group squaring (i.e., point doubling in Jacobian coordinates), Algorithm 3.21 from [18] requires 4 field multiplications and 4 field squarings (ignoring the cost of field additions and multiplications by small constants).

Exponentiation without precomputation (i.e., scalar multiplication with non-fixed base) is more expensive by orders of magnitude. Algorithm 3.27 from [18], which is the elliptic curve analogue of repeated square-and-multiply, requires 256 group squarings and ~ 128 group multiplications when applied for P-256; given the previously fixed costs, this is the equivalent of 2560 field multiplications and 1536 field squarings in Jacobian coordinates.

Hashing is an entirely different operation, based on sequential byte processing rather than parallelizable arithmetic; it is treated as non-comparable cost and usually ignored in elliptic curve cost modeling. However, hash-into-field is invoked $O(k)$ times in OSST, so it makes sense to include it in our estimations. We model its cost based on SHA-256. This operation requires ~ 10 cycles per byte for short messages on a x86-64 processor without dedicated extensions. Assuming that a 256-bit modular reduction requires ~ 5 cycles, hashing a P-256 point (94 bytes) into field requires ~ 945 cycles. Assuming that Montgomery multiplication requires ~ 13 cycles, this is the equivalent of ~ 70 field multiplications in total. This ratio falls significantly if the CPU comes with SHA-NI extensions, in which case hashing a short message costs ~ 2 -4 cycles per byte.

Assuming that field squaring costs 0.8α , we conclude with Table 3 for working purposes.

Table 3: Fixed relative time costs for P-256 over x86-64

Operation		P-256/x86-64
Field multiplication	(α/α)	1
Modular inversion	(β/α)	80
Field addition/subtraction	(γ/α)	0
Group multiplication	(δ/α)	15.2
Group squaring	(ϵ/α)	7.2
Exponentiation	(ζ/α)	3788.8
Hashing into field	(η/α)	70

6.2 Multi-exponentiation algorithms

A *multi-exponentiation* is an algorithm that evaluates the product

$$\prod_{i=1}^k g_i^{z_i},$$

⁹ Implementations based on Fermat’s Little Theorem require ~ 255 squarings and 11 multiplications, yielding a significantly higher cost of $\sim 255 \cdot 0.8 + 11 \approx 215$ multiplications.

given $(g_i, z_i) \in \mathbb{G} \times \mathbb{Z}_q$ for $i = 1, \dots, k$ and $k \geq 1$. Naive iteration is remarkably slow even for small k 's due to the high cost of exponentiation; we are thus interested in algorithms that operate on the binary representations of the exponents efficiently. Their efficiency relies on tunable precomputation tables that accelerate the evaluation phase by reducing the number of required group operations; the lifetime of these tables extends –roughly speaking– as long as the same set of base elements can be reused. Below we consider the two multi-exponentiation algorithms that are widely implemented and used in cryptographic applications. We denote

$$b \equiv \log q$$

the bitlength of the group order, which for our case is the uniform bitlength of the exponents.

The wNAF-based interleaving algorithm (cf. [24]) is applicable whenever group inversion is cheap (as is the case with the elliptic curves). It generalizes the sliding window method by assigning distinct windows to distinct bases, which means that every base element has its own precomputation table. This makes it particularly apt for incremental verification because the same bases are repeatedly used, with an extra commitment added per incoming connection, and the precomputation overhead is thus amortized across the session. However, its efficiency depends crucially on how fast the multiple distinct tables are accessed in memory in a random pattern; consequently, the precomputation parameters must be chosen carefully so that the required memory does not overly exceed the space allocated in the CPU cache for the precomputed tables per session. The algorithm has linear time complexity and becomes infeasible for very large k 's, even though practical efficiency can possibly be expected in the range $100 < k < 500$ with careful tuning on x86-64 platforms.

Pippenger's algorithm (cf. [28], [9]) remains state-of-the-art for excessively large k 's due to its $O(bk/\log(bk))$ time complexity. Similar to wNAF-based interleaving, the precomputed buckets depend only on the base elements, allowing reusability with a different set of exponents. The algorithm operates with a uniform window that is chosen to be around

$$w \sim \log(bk) - 2 \log \log(bk)$$

but must be calibrated per CPU by profiling. Concretely, the algorithm requires cache storage for $\sim 2^w$ precomputed group elements, involves $\sim 2^w + bk/w$ group operations that dominate execution time for large k 's and has better memory access patterns. It should be stressed, however, that every base corresponds to several buckets, meaning that precomputation must be repeated from scratch even if a single base element is omitted, replaced or added. This makes it less flexible with respect to incremental verification; unless the precomputed buckets could be updated in a cheap way per incoming connection, the algorithm should only be used if the reduction in the main evaluation runtime pays off for the upfront costs of repeated precomputation. Moreover, its practical efficiency may be negatively affected by the structural dependencies among the weights; specifically, the shared algebraic substructure of $\bar{c}, \mu_{i_1}, \dots, \mu_{i_k}$ (cf. (3.4)) may reduce the bucket reuse opportunities that the algorithm exploits in the case of completely independent exponents. Even so, the algorithm is expected to vastly outperform wNAF-based interleaving beyond a cross-over point well below $k = 500$.

The transition zone between wNAF-based interleaving and Pippenger's algorithm needs to be determined per device by means of profiling. In what follows, we focus on the effect of interleaving on the performance of OSST verifiers.

6.2.1 The wNAF-based Interleaving Exponentiation Algorithm

We denote by $\tau \leftarrow \text{wNAF-Table}(u, w)$, $u \in \mathbb{G}$, $w \in \mathbb{Z}_+$, the subroutine that precomputes the wNAF-table for a given base and window. We also denote by

$$\text{wNAF-Exp}(g_0, g_1, \dots, g_k; z_0, z_1, \dots, z_k; \tau_0, \tau_1, \dots, \tau_k)$$

the main evaluation stage of the algorithm, accepting as arguments the totality of base elements $g_0, \dots, g_k$ along with the respective exponents $z_0, \dots, z_k$ and precomputed tables $\tau_\nu \equiv \text{wNAF-Table}(g_\nu, w_\nu)$, $0 \leq \nu \leq k$. In this notation, OSST verification (3.3) translates to

$$\exp\left(g, \sum_{\nu=1}^k \mu_{i_\nu} s_{i_\nu}\right) = \text{wNAF-Exp}(y, u_{i_1}, \dots, u_{i_k}; \bar{c}, \mu_{i_1}, \dots, \mu_{i_k}; \tau_0, \tau_{i_1}, \dots, \tau_{i_k}), \quad (6.1)$$

$Q = \{i_1, \dots, i_k\}$, where $\tau_0 = \text{wNAF-Table}(y, w_0)$, $\tau_{i_\nu} = \text{wNAF-Table}(u_{i_\nu}, w)$, $\nu = 1, \dots, k$, and the uniform window w remains to be justified. Below we follow closely the analysis from [24], §3, in order to estimate the costs of evaluating the right hand side of (6.1).

Since the exponents have equal bitlength, the windows need only be chosen according to the lifetime of the respective bases. Recall that a larger window saves time during the main evaluation phase at the cost of a larger table and longer precomputation. Since y is used across many identification sessions, it makes sense to take a larger w_0 and store τ_0 persistently alongside y ; this requires a computation time of $(2^{w_0-1} - 1)\delta + \epsilon$ and a persistent storage of $(2^{w_0-1} - 1)\mathcal{U}$. The commitments do not outlive the session and are repeatedly used for the purpose of incremental verification (everytime with one more base element and fresh exponents); consequently, their tables must be stored in the main memory, with one table added upon every accepted connection. Each table requires a computation time of

$$T_{\text{wNAF-Table}}(w) = (2^{w-1} - 1)\delta + \epsilon \quad (6.2)$$

while the storage required for it in the main memory is

$$S_{\text{wNAF-Table}}(w) = (2^{w-1} - 1)\mathcal{U}. \quad (6.3)$$

The total working memory requirement for the precomputed tables is namely

$$S_{\text{wNAF-Exp}}^{\text{prec}} = [(2^{w_0-1} - 1) + k \cdot (2^{w-1} - 1)]\mathcal{U}. \quad (6.4)$$

Ideally, this should not generally exceed the effective L1/L2 cache capacity while k increases in order for the effect of slow memory access during the main evaluation phase of the algorithm to be minimized. This is the reason for choosing a uniform and relatively small window w .

We proceed to the main evaluation phase. Ignoring bitwise operations, NAF conversion requires $\sim 2b \cdot (1/(w_0 + 2) + k/(w_0 + 2))$ field subtractions (two per non-zero digit with density $\sim 1/(w_0 + 2)$ for the first array and density $\sim 1/(w + 2)$ for the rest). The main loop requires b group squarings and $\sim (1/(w_0 + 2) + k/(w + 2))$ group multiplications per iteration (again due to the same densities). We conclude with an ideal expected time cost of

$$T_{\text{wNAF-Exp}}(k; w_0, w) = b \cdot \left(\frac{1}{w_0 + 2} + k \frac{1}{w + 2} \right) \cdot (\delta + 2\gamma) + b\epsilon, \quad (6.5)$$

which makes apparent how larger windows save execution time. Regarding storage needs, NAF conversion requires $k + 1$ arrays of length $b + 1$; the first array contains integers in the range $[\pm 2^{w_0+1} - 1]$ while the rest from $[\pm 2^{w+1} - 1]$, yielding a peak memory usage equal to

$$S_{\text{wNAF-Exp}}(k; w_0, w) = \frac{b+1}{8} [(w_0 + 1) + k(w + 1)] \quad (\text{bytes}). \quad (6.6)$$

Effective execution time on x86-64/ARM64 Every group multiplication contributing to the first summand of (6.5) involves an element from the precomputed tables. Iteratively fetching these elements follows a random memory access pattern because the sequence of tables is determined by the unpredictable distribution of non-zero digits in the NAF arrays. From the CPU cache perspective, this means that the algorithm has poor spatial and temporal locality; while k increases, cache contention increases uniformly across the whole range of precomputed tables, meaning that an already fetched element has increasingly more chances to be evicted to a slower memory layer before being reused. Since the number of iterations grows excessively with k by a factor of $b/(w + 2)$, the slow memory effect incurs hidden costs that are non-negligible and, in fact, dominate execution time for large k 's. In order to capture these costs, the effective execution time of wNAF-Exp is better modelled as

$$T_{\text{wNAF-Exp}}^{\text{eff}}(k; w_0, w) = b \cdot \left(\frac{1}{w_0 + 2} + k \frac{1}{w + 2} \right) \cdot (\delta + L_{\text{mem}}(k; w_0, w) + 2\gamma) + b\epsilon,$$

where the non-linear factor $L_{\text{mem}}(k; w_0, w)$ measures the expected latency of accessing the desired element upon each iteration. This is a highly indeterminate variable; beyond the working memory needs (cf. (6.4)) and the latencies of memory hierarchy, it depends on unknown system variables that affect cache contention (e.g., scheduling of concurrent processes and the relevant context switches, unknown memory access patterns, OS overhead etc.) and can only be determined empirically through profiling (assuming a relatively stable load on the operating system). In particular, formula (6.5) fails to capture the saturation point beyond which random memory access causes serious performance bloat and Pippenger's algorithm becomes significantly more efficient. This discussion is relevant for the x86-64 and ARM64 architectures; on AVR platforms, feasibility depends entirely on whether the precomputed tables fit within the limited SRAM of the microcontroller.

6.3 Storage infrastructure

Given an array a of length $l \geq 1$, we denote by $a[\nu]$ its ν -th element, $1 \leq \nu \leq l$. We write

$$a \leftarrow l * [x]$$

to indicate the initiation of an array a of length l filled with x 's. Given an array a^* , we write

$$a \leftarrow a^*$$

to indicate buffer reuse, i.e., the initiation of an array a occupying the same memory segment as a^* . Arrays are implicitly assumed to be dropped when not anymore needed in order for the peak memory usage of the various algorithms to be minimized.

Given a vector v of length $l \geq 0$, we similarly denote by $v[\nu]$ its ν -th element, $1 \leq \nu \leq l$. Appending an element x to the vector v is denoted by

$$v += [x]$$

while removing the same element from the end of the array is denoted by

$$v -= [x].$$

6.3.1 Fixed storage needs

For every session, the verifier initiates empty vectors $\hat{u}$, $\hat{s}$, $\hat{q}$ for storing the individual proof packets. Specifically, for every incoming packet (u_i, s_i) , the verifier appends u_i, s_i to $\hat{u}$ and $\hat{s}$ respectively, keeping track of the respective share index i in $\hat{q}$; concretely,

$$\hat{u}[\nu] = u_{i_\nu}, \quad \hat{s}[\nu] = s_{i_\nu}, \quad \hat{q}[\nu] = i_\nu,$$

where the packets (u_{i_ν}, s_{i_ν}) , $\nu = 1, 2, \dots$ come in this order. Assuming that each index occupies 2 bytes of memory, the total memory requirement for k aggregated proof packets is

$$(\mathcal{U} + \mathcal{S} + 2)k \tag{6.7}$$

in the notation of Table 1b (e.g., $128k$ bytes for P-256 over x86-64, cf. Table 2). This cost is manageable even for very large k 's on x86-64 or ARM64 architectures; on AVR platforms, a relatively low bound must be carefully set on the number of admissible packets according to the stringent memory constraints of the microcontroller. The storage cost for $\hat{u}, \hat{s}, \hat{q}$ will be ignored in the comparative analysis of naive and optimized verification because it is common in both of them; however, it must be taken into account when estimating the absolute memory requirement for verification in any of its variants.

6.3.2 Working set of precomputed wNAF-tables

When applying multi-exponentiation (cf. Section 6.2), the verifier must accommodate storage for the precomputed tables in the main memory. In the case of wNAF-based interleaving, the total memory requirement is given by (6.4), accounting for τ_0 (which is persistently stored alongside y) and the k precomputed tables for the commitments. Concretely, the verifier initiates an empty vector $\hat{\tau}$ for appending the wNAF-table of u_i , i.e.,

$$\hat{\tau}[\nu] = \text{wNAF-Table}(\hat{u}[\nu]), \tag{6.8}$$

assuming that the packets (u_{i_ν}, s_{i_ν}) , $\nu = 1, 2, \dots$ come in this order. In this notation, the right-hand side of the verification condition (6.1) translates to

$$\text{wNAF-Exp}(y, \hat{u}[1], \dots, \hat{u}[k]; \bar{c}, \hat{\mu}[1], \dots, \hat{\mu}[k]; \tau_0, \hat{\tau}[1], \dots, \hat{\tau}[k]),$$

where $\hat{\mu}$ is an array holding the weights $\{\mu_{i_\nu}\}_{\nu=1}^k$, i.e., $\hat{\mu}[\nu] = \mu_{i_\nu}$. In the following section, we describe an efficient way for computing this array.

6.4 Weights computation

Iteratively computing the right-hand side of (3.3) with naive exponentiation does not require the weights $\{\mu_i\}_{i \in Q}$ to be in memory simultaneously; these can be computed lazily along with the respective interpolation coefficients. Nevertheless, the total cost of their computation accumulates irrespective of this fact, including the total cost of computing $\{\lambda_i\}_{i \in Q}$. This is

$$T_{\text{LagrNaive}}(k) = (2\alpha + \beta + \gamma)k^2 + (\beta - \alpha + \gamma)k \quad (6.9)$$

for $k = |Q|$ (cf. (B.5)), which incurs significant latency due to βk^2 . Computing the interpolation coefficients in more efficient ways comes at the cost of non-trivial space complexity. Nevertheless, multi-exponentiation algorithms (cf. Section 6.2) require the weights to be in memory simultaneously; since each weight is essentially a scaling of the respective interpolation coefficient, the unavoidable space complexity of computing the interpolation coefficients efficiently can be exploited in the context of optimized verification by means of buffer reuse.

6.4.1 Common denominator technique

We recall an efficient technique for computing the interpolation coefficients for a given set $Q \subset \{1, \dots, n\}$, $k = |Q|$. The algorithm has $O(k^2)$ complexity but the constant factors are very small because it involves only one modular inversion. This comes at the cost of $O(k)$ space complexity, which can later be exploited for holding the weights in the main memory.

Note first that the interpolation coefficients $\{\lambda_i\}_{i \in Q}$ (cf. B.5) can be written in the form

$$\lambda_i = \xi \cdot d_i^{-1}, \quad d_i \equiv i \prod_{j \in Q \setminus \{i\}} (j - i), \quad i \in Q, \quad \xi \equiv \prod_{j \in Q} j.$$

This still requires k modular inversions for the elements $\{d_i\}_{i \in Q}$; however, their inverses can be written in the form

$$d_i^{-1} = \rho_i \cdot \bar{d}^{-1}, \quad \rho_i \equiv \prod_{j \in Q \setminus \{i\}} d_j, \quad i \in Q, \quad \bar{d} \equiv \prod_{i \in Q} d_i$$

which requires a single inversion for $\bar{d}$. We conclude that

$$\lambda_i = \xi \cdot \rho_i \cdot \bar{d}^{-1}, \quad \xi \equiv \prod_{j \in Q} j, \quad \rho_i \equiv \prod_{j \in Q \setminus \{i\}} d_j, \quad i \in Q, \quad \bar{d} \equiv \prod_{i \in Q} d_i, \quad (6.10)$$

where

$$d_i \equiv i \prod_{j \in Q \setminus \{i\}} (j - i), \quad i \in Q. \quad (6.11)$$

The exact computation is shown in Algorithm 6, assuming $Q = \{i_1, \dots, i_k\}$ with $i_\nu = \hat{q}[\nu]$ for $\nu = 1, \dots, k$. Observe that it employs a forward-backward pass for computing $\rho_{i_1}, \dots, \rho_{i_k}$ in order to further reduce the number of required field multiplications. Specifically, it requires $k^2 + 4k$ field multiplications, 1 modular inversion and $k(k - 1)$ subtractions; in the notation of Table 1a, this yields an estimated execution time equal to

$$T_{\text{LagrCommd}}(k) = (\alpha + \gamma)k^2 + (4\alpha - \gamma)k + \beta \quad (6.12)$$

Ingoring constant overhead terms, the peak memory usage is

$$S_{\text{LagrCommd}}(k) = (2k + 1)\mathcal{S} \quad (6.13)$$

in the notation of Table 1b. This can be insignificant even for very large k 's on x86-64 or ARM64; on AVR, however, it potentially puts a rather low bound even on well-equipped MCUs (8–32 KB RAM for cryptographic operations). For x86-64, the performance advantage is vast as shown in Table 4, where we have plugged the values from Tables 2 and 3.

Algorithm 6 $\text{LagrCommd}(\hat{q})$

```

1:  $\xi \leftarrow 1$ 
2: for  $\nu = 1, \dots, k$ 
3:    $\xi \leftarrow \hat{q}[\nu] \cdot \xi$ 
4:  $\hat{d} \leftarrow k * [1]$ 
5: for  $\nu = 1, \dots, k$ 
6:    $i \leftarrow \hat{q}[\nu]$ 
7:    $d \leftarrow i$ 
8:   for  $\mu = 1, \dots, k$ 
9:     if  $\mu = \nu$ 
10:      continue
11:      $j \leftarrow \hat{q}[\mu]$ 
12:      $d \leftarrow d \cdot (j - i)$ 
13:    $\hat{d}[\nu] \leftarrow d$ 
14:  $\hat{\rho} \leftarrow k * [1]$ 
15: for  $\nu = 2, \dots, k$ 
16:    $\hat{\rho}[\nu] \leftarrow \hat{\rho}[\nu - 1] \cdot \hat{d}[\nu - 1]$ 
17:  $\bar{d} \leftarrow 1$ 
18: for  $\nu = k, \dots, 1$ 
19:    $\hat{\rho}[\nu] \leftarrow \hat{\rho}[\nu] \cdot \bar{d}$ 
20:    $\bar{d} \leftarrow \hat{d}[\nu] \cdot \bar{d}$ 
21:  $\hat{\lambda} \leftarrow \hat{\rho}$ 
22:  $\Delta \leftarrow \xi \cdot \bar{d}^{-1}$ 
23: for  $\nu = 1, \dots, k$ 
24:    $\lambda[\nu] \leftarrow \Delta \cdot \rho[\nu]$ 
25: return  $\hat{\lambda}$ 

```

6.4.2 Normalizer and weights

By definition of weights (cf. Definition (3.7)), given the interpolation coefficients $\{\lambda_i\}_{i \in Q}$ for a set $Q \subset \{1, \dots, n\}$, we need an efficient way to compute the scaling factors

$$\prod_{j \in Q \setminus \{i\}} c_j, \quad i \in Q,$$

Looping requires too many field multiplications, incurring latency for large k 's, while computing the weights as $\bar{c} \cdot c_i^{-1}$, $i \in Q$, requires k modular inversions. Since hashing is faster by order of magnitude, it makes sense to avoid inversions and apply a forward-backward pass.

The exact computation is shown in Algorithm 7. Except for invoking `LagrCommd`, it requires $4k - 4$ field-multiplications and $2k$ hashes; in the notation of Table 1a, this yields an estimated execution time equal to

$$T_{\text{WeightsCommd}}(k) = T_{\text{LagrCommd}}(k) + (4\alpha + 2\eta)k + (2\eta - 4\alpha), \quad (6.14)$$

(cf. (6.12)). Peak memory is during the computation of the interpolation coefficients, so that

$$S_{\text{WeightsCommd}}(k) = S_{\text{LagrCommd}}(k) = (2k + 1)\mathcal{S} \quad (6.15)$$

(cf. (6.13)). The estimated execution time of the algorithm is dominated by the computation of interpolation coefficients as shown in the middle columns of Table 4, where we have plugged the values from Tables 2 and 3.

Algorithm 7 `WeightsCommd`($\hat{u}, \hat{q}$)

```

1:  $\hat{\lambda} \leftarrow \text{LagrCommd}(\hat{q})$ 
2:  $c^* \leftarrow H(\hat{u}[1])$ 
3:  $\hat{\mu} \leftarrow \hat{\lambda}$ 
4: for  $\nu = 2, \dots, k$ 
5:    $\hat{\mu}[\nu] \leftarrow c^* \cdot \hat{\lambda}[\nu]$ 
6:    $c^* \leftarrow H(\hat{u}[\nu]) \cdot c^*$ 
7:  $\bar{c} \leftarrow c^*$ 
8:  $c^* \leftarrow H(\hat{u}[k])$ 
9: for  $\nu = k - 1, \dots, 1$ 
10:   $\hat{\mu}[\nu] \leftarrow c^* \cdot \hat{\mu}[\nu]$ 
11:   $c^* \leftarrow H(\hat{u}[\nu]) \cdot c^*$ 
12: return  $\hat{\mu}, \bar{c}$ 

```

Table 4: Weights computation costs for P-256/x86-64 (3.5 GHz)

k	$T_{\text{LagrNaive}}$ (ms)	$T_{\text{LagrCommd}}$ (ms)	$T_{\text{WeightsCommd}}$ (ms)	$S_{\text{WeightsCommd}}$ (KB)
5	0.245	0.013	0.063	0.344
10	0.899	0.022	0.109	0.656
20	3.438	0.056	0.216	1.281
50	20.895	0.278	0.657	3.156
100	82.790	1.048	1.792	6.281
200	329.580	4.088	5.562	12.531
500	2053.950	25.208	28.872	31.281
1000	8207.900	100.408	107.722	62.531

Table 5: Naive verification costs for P-256/x86-64 (3.5 GHz)

k	$T_{\text{WeightsCommd}} / T_{\text{VerifyNaive}}$	$T_{\text{VerifyNaive}}$ (ms)
5	0.023	2.723
10	0.023	4.671
20	0.025	8.583
50	0.032	20.439
100	0.044	40.599
200	0.067	82.419
500	0.131	219.879
1000	0.220	488.979

6.5 Core verification routines

We proceed to the comparative analysis of naive and optimized verification. As expected, wNAF-based verification is found to be ~ 3 -5x faster than the naive approach, which is based on iterative exponentiation; in practice, however, the actual advantage is lower for, say, $k \geq 100$ due to the effect of random memory access (cf. Section 6.2.1). Even more, the performance of wNAF-based verification is expected to bloat beyond an empirically detected saturation point, where precomputed wNAF-tables reside effectively in the main memory and Pippenger’s algorithm becomes appropriate despite its inflexible precomputation stage.

6.5.1 Naive verification

Naive verification is shown in Algorithm 8. Apart from computing weights, it involves k field multiplications, k field additions $k + 2$ exponentiations and k group multiplications; in the notation of Table 1a, this yields an estimated execution time equal to

$$T_{\text{Naive-Verify}}(k) = T_{\text{WeightsCommd}}(k) + (\alpha + \gamma + \delta + \zeta)k + 2\zeta, \quad (6.16)$$

(cf. (6.14)). Despite being $O((\alpha + \gamma)k^2)$, this runtime is dominated by ζk because $\zeta \gg \alpha + \gamma$. Nevertheless, as shown in the first column of Table 5, the optimized computation of weights contributes significantly, indicating that the effect of naively computing the interpolation

coefficients would be substantial and, in fact, prohibitive; as a consequence, the same holds true for optimized verification (cf. Section 6.5.2), where the total execution time is expected to be still lower. The peak memory usage of the algorithm is attained during the computation of weights and thus coincides with (6.15).

Algorithm 8 Naive-Verify ($\hat{u}, \hat{s}, \hat{q}; y$)

```

1:  $\hat{\mu}, \bar{c} \leftarrow \text{WeightsCommd}(\hat{u}, \hat{q})$ 
2:  $\bar{s} \leftarrow 0, \bar{u} \leftarrow 1,$ 
3: for  $\nu = 1, \dots, k$ 
4:    $\mu \leftarrow \hat{\mu}[\nu]$ 
5:    $\bar{s} \leftarrow \bar{s} + \mu \cdot \hat{s}[\nu]$ 
6:    $\bar{u} \leftarrow \bar{u} \cdot \hat{u}[\nu]^\mu$ 
7:  $\text{lhs} \leftarrow g^{\bar{s}}$ 
8:  $\text{rhs} \leftarrow y^{\bar{c}} \cdot \bar{u}$ 
9: return  $\text{lhs} = \text{rhs}$ 

```

6.5.2 Optimized verification

wNAF-based verification is shown in Algorithm 9, where τ_0 stands for the precomputed table of y and $\hat{\tau}$ is a vector holding the precomputed tables of the commitments (cf. (6.8)). Apart from invocations, it involves k field multiplications, k field additions and 1 exponentiation; in the notation of Table 1a, this yields an estimated execution time equal to

$$T_{\text{wNAF-Verify}}(k; w_0, w) = T_{\text{WeightsCommd}}(k) + T_{\text{wNAF-Exp}}(k; w_0, w) + (\alpha + \gamma)k + \zeta, \quad (6.17)$$

(cf. (6.14) and (6.2.1)), where w_0 is the window size for y and w is the uniform window of the commitments (cf. Section 6.2.1). This does not include the cost of precomputation, which is

$$T_{\text{wNAF-Verify}}^{\text{prec}}(k; w) = k \cdot T_{\text{wNAF-Table}}(w), \quad (6.18)$$

(cf. (6.2)), assuming that the table of y is precomputed once and persistently stored upon key registration. The maximum storage need after computing weights is

$$S_{\text{wNAF-Verify}}(k; w_0, w) = S_{\text{wNAF-Exp}}(k; w_0, w) + k\mathcal{S} + \mathcal{U} \quad (6.19)$$

(cf. (6.6)), where $k\mathcal{S}$ stems from the array $\hat{\mu}$ and the summand $\mathcal{U}$ from $g^{\bar{s}}$. It is easy to reason that this sum is larger than (6.15) for every practical value of w , so that $S_{\text{wNAF-Verify}}$ represents indeed the peak memory usage of the algorithm. This does not include the storage need for the totality of precomputed tables, which is

$$S_{\text{wNAF-Verify}}^{\text{prec}}(k; w_0, w) = S_{\text{wNAF-Exp}}^{\text{prec}}(k; w_0, w) \quad (6.20)$$

(cf. (6.4)). In this notation, the peak memory requirement for wNAF-based verification is

$$S_{\text{wNAF-Verify}}^*(k; w_0, w) = S_{\text{wNAF-Verify}}^{\text{prec}}(k; w_0, w) + S_{\text{wNAF-Verify}}(k; w_0, w) \quad (6.21)$$

where the second term is given by (6.19). Some concrete estimates are shown in Table 6, where we have plugged the values from Tables 2 and 3. It should be stressed again that the first columns of 6a-6c become misleading beyond a saturation point that needs to be empirically determined per device by means of profiling; concretely, this is the k value beyond which the working set shown in the third column resides effectively in the main memory. Finally, the last column does not account for the fixed storage needs of verification (cf. (6.7)).

Algorithm 9 $\text{wNAF-Verify}(\hat{u}, \hat{s}, \hat{q}, \hat{\tau}; y, \tau_0)$

```

1:  $\hat{\mu}, \bar{c} \leftarrow \text{WeightsCommd}(\hat{u}, \hat{q})$ 
2:  $\bar{s} \leftarrow 0$ 
3: for  $\nu = 1, \dots, k$ 
4:    $\bar{s} \leftarrow \bar{s} + \hat{\mu}[\nu] \cdot \hat{s}[\nu]$ 
5:  $\text{lhs} \leftarrow g^{\bar{s}}$ 
6:  $\text{rhs} \leftarrow \text{wNAF-Exp}(y, \hat{u}[1], \dots, \hat{u}[k]; \bar{c}, \hat{\mu}[1], \dots, \hat{\mu}[k]; \tau_0, \hat{\tau}[1], \dots, \hat{\tau}[k])$ 
7: return  $\text{lhs} = \text{rhs}$ 

```

6.6 Incremental verification

Suppose that a baseline coalition has been identified, with their proof packets appended to the vectors $\hat{u}, \hat{s}, \hat{q}$ (cf. Section 6.3.1) and the wNAF-tables for the commitments appended to $\hat{\tau}$ (cf. Section 6.3.2). Incremental verification consists in the following loop, where (u, s, j) is the concrete representation for an incoming packet (u_j, s_j) .

```

while incoming connection  $(u, s, j)$ 
   $\tau \leftarrow \text{wNAF-Table}(u)$ 
   $\hat{u} += [u], \hat{s} += [s], \hat{q} += [j], \hat{\tau} += [\tau]$ 
  if  $\text{wNAF-Verify}(\hat{u}, \hat{s}, \hat{q}, \hat{\tau}; y, \tau_0) = \text{true}$ 
    ... // Accept connection
  else
     $\hat{u} -= [u], \hat{s} -= [s], \hat{q} -= [j], \hat{\tau} -= [\tau]$ 
    ... // Drop connection

```

Omitting the latency from network I/O and updating the vectors, the purely cryptographic execution time of the k -th iteration is

$$T_{\text{wNAF-Verify}}^{\text{inc}}(k+1; w_0, w) = T_{\text{wNAF-Table}}(w) + T_{\text{wNAF-Verify}}(k+1; w_0, w), \quad (6.22)$$

where $k \geq t$ is the number of aggregated proof packets excluding the currently incoming one. Practical efficiency is demonstrated in Table 7; this, however, becomes misleading beyond a saturation point that needs to be empirically detected per device through profiling.

Table 6: wNAF-based verification costs for P-256/x86-64 (3.5 GHz)

(a) $w_0 = 5, w = 2$

k	$T_{\text{wNAF-Verify}}$ (ms)	$T_{\text{wNAF-Verify}}^{\text{prec}}$ (ms)	$S_{\text{wNAF-Verify}}^{\text{prec}}$ (KB)	$S_{\text{wNAF-Verify}}^*$ (KB)
5	1.168	0.011	1.836	2.743
10	1.701	0.022	2.295	3.829
20	2.782	0.045	3.213	6.000
50	6.144	0.112	5.967	12.515
100	12.148	0.224	10.557	23.373
200	25.656	0.448	19.736	45.090
500	78.180	1.120	47.275	110.239
1000	205.720	2.240	93.174	218.820

(b) $w_0 = 5, w = 3$

k	$T_{\text{wNAF-Verify}}$ (ms)	$T_{\text{wNAF-Verify}}^{\text{prec}}$ (ms)	$S_{\text{wNAF-Verify}}^{\text{prec}}$ (KB)	$S_{\text{wNAF-Verify}}^*$ (KB)
5	1.071	0.026	2.754	3.818
10	1.507	0.053	4.131	5.978
20	2.393	0.106	6.885	10.300
50	5.172	0.264	15.147	23.263
100	10.203	0.528	28.916	44.870
200	21.765	1.056	56.455	88.083
500	68.452	2.640	139.072	217.721
1000	186.264	5.280	276.768	433.786

(c) $w_0 = 5, w = 5$

k	$T_{\text{wNAF-Verify}}$ (ms)	$T_{\text{wNAF-Verify}}^{\text{prec}}$ (ms)	$S_{\text{wNAF-Verify}}^{\text{prec}}$ (KB)	$S_{\text{wNAF-Verify}}^*$ (KB)
5	0.960	0.118	8.262	9.639
10	1.284	0.235	15.147	17.621
20	1.948	0.470	28.916	33.586
50	4.060	1.176	70.225	81.479
100	7.979	2.352	139.072	161.301
200	17.318	4.704	276.768	320.944
500	57.335	11.760	689.854	799.875
1000	164.029	23.520	1378.330	1598.090

Table 7: Incremental wNAF-based verification costs for P-256/x86-64 (3.5 GHz)

k	$T_{\text{wNAF-Verify}}^{\text{inc}}(\cdot; 5, 2)$ (ms)	$T_{\text{wNAF-Verify}}^{\text{inc}}(\cdot; 5, 3)$ (ms)	$T_{\text{wNAF-Verify}}^{\text{inc}}(\cdot; 5, 5)$ (ms)
2	0.959	0.903	0.855
5	1.277	1.163	1.048
10	1.811	1.600	1.373
20	2.893	2.488	2.039
50	6.262	5.273	4.157
100	12.276	10.314	8.086
200	25.804	21.896	17.445
500	78.388	68.643	57.522
1000	206.028	186.555	164.316

7 Conclusion

We have investigated the overlooked area of real-time threshold identification, arguing that threshold signing primitives are not native solutions unless adapted to the permissionless setting, where verifiers must play the role of combiner. In this context, we have stressed that practical schemes for open decentralized networks cannot rely on advertized shares because these are generally infeasible to attest, certify and monitor; at the same time, real-time interaction among the provers appears as an inconvenience –if not an operational obstacle– without essential gains for identification. In particular, we have argued that share-free and non-interactive schemes emerge as the most native and system-agnostic option for ideal open decentralized meshes (i.e., lacking ad hoc infrastructure and governance), implicitly treating this “niche” as canonical while threshold cryptography has focused on the closed committee model under idealized assumptions regarding its infrastructure.

We have introduced OSST as a concrete and lightweight instance of such a protocol in the pure DL-setting, proving a certain security notion under OMDL hardness and anticipating deployability through a back-of-the-envelope calculation. Regarding security, we have seen that a formal proof is in principle possible without the assumption of trusted shares, if at the cost of restricting genuinely our initial attack model. Specifically, we have confined ourselves to a reasonably narrow threat model that allows extractability under the only currently available extraction formula for the proposed protocol, leaving room for refinements that may facilitate a security proof in the most general adversarial setting.

A Schnorr identification

Let $\mathbb{G} = \langle g \rangle$ be a cyclic group of order q . A secret key is a scalar $x \in \mathbb{Z}_q$ and $y = g^x$ is its public counterpart; equivalently, $x = \log y$, where the logarithm is taken with respect to g . *Discrete-logarithm hardness* (DL) for the system under consideration is the assumption that computing the logarithm of a uniformly random group element is infeasible for every existing computer. More accurately, given $y \leftarrow_{\$} \mathbb{G}$, every probabilistic polynomial-time algorithm should be able to compute $x \equiv \log y$ with negligible chance of success, where negligibility is determined by the current level of disposable computational power in the security parameter.

The *Schnorr protocol* is a non-interactive zero-knowledge (NIZK) scheme, allowing the holder of $y = g^x$ to prove knowledge of x without revealing any information about the latter.

Its original interactive variant [30] is a sigma protocol. The prover samples $r \leftarrow_{\$} \mathbb{Z}_q$, stores it for the current session and sends the commitment $u \leftarrow g^r$ to the verifier. The verifier samples a challenge $c \leftarrow_{\$} \mathbb{Z}_q$, caches it for the current session and sends it to the prover. Finally, the prover responds with $s = r + cx$ and the verifier accepts only if the relation

$$g^s = y^c u$$

is satisfied; indeed, $g^s = g^{r+cx} = g^{xc} g^r = y^c u$.

Given a hash function $H : \mathbb{G} \rightarrow \mathbb{Z}_q$, the protocol can be made non-interactive through the Fiat-Shamir transform; the prover computes $c \leftarrow H(u)$ and sends (u, s) to the verifier, who retrieves c from u and proceeds to verification. This is preferable due to greater robustness and reduced bandwidth. Intuitively, the hash function injects sufficiently high entropy across every bit of its output, so that assigning $c \leftarrow H(u)$ becomes indistinguishable from uniformly sampling c . The current mainstream formalizes this fact in the random oracle model (ROM).

The protocol identifies any procedure that “sees” the logarithm of a given group element. Security against impersonation attacks means that it is infeasible to forge a valid proof for a group element without knowing its logarithm; in the proof-of-knowledge framework, this is formalised as witness extractability under the DL hardness assumption. The protocol is meaningful only under DL hardness, which in turn enables a security proof in the ROM.¹⁰

B Shamir’s Secret Sharing

We here summarize the basic facts regarding the sharing scheme introduced by [32]. Given a cyclic group $\mathbb{G}$ of prime order q , we denote by $(\mathbb{Z}_q)_{t-1}[X]$ the ring of polynomials of degree at most $t-1$ over $\mathbb{Z}_q$, $t \geq 1$.

Definition B.1. The *Shamir (n, t) -sharing scheme*, $1 \leq t \leq n < q$, is the probabilistic algorithm $(x_1, \dots, x_n; y) \leftarrow D(g, x)$, $(g, x) \in \mathbb{G} \times \mathbb{Z}_q$, defined as

$$y \leftarrow g^x, \quad x_i \leftarrow f(x_i), \quad 1 \leq i \leq n, \quad f \leftarrow_{\$} (\mathbb{Z}_q)_{t-1}[X] \text{ with } f(0) = x$$

Definition B.2. For fixed $g \in \mathbb{G}$, the space of Shamir (n, t) -sharings of x is denoted by

$$D_x = \{(x_1, \dots, x_n; y) \mid (x_1, \dots, x_n; y) \leftarrow D(g, x)\}$$

Remark B.3. (*Interpolation formula*) Given $(x_1, \dots, x_n; y) \in D_x$, the condition

$$x = \sum_{i \in Q} \lambda_i x_i \iff |Q| \geq t \iff y = \prod_{i \in Q} y_i^{\lambda_i}, \quad (\text{B.4})$$

holds true, where $\{\lambda_i\}_{i \in Q}$ are the Lagrange interpolation coefficients for Q , i.e.,

$$\lambda_i = \prod_{j \in Q \setminus \{i\}} j(j-i)^{-1}, \quad i \in Q, \quad (\text{B.5})$$

and $y_i \equiv g^{x_i}$, $1 \leq i \leq n$, are the public shares.

¹⁰ The most far-reaching result in this direction is security against active and concurrent impersonation attacks under one-more discrete logarithm (OMDL) hardness; cf. Theorem 2, [8].

Remark B.6. (*Homomorphic property*) By the properties of polynomial evaluation, adding

$$(x_1, \dots, x_n; y) \in D_x, (x_1^*, \dots, x_n^*; y^*) \in D_{x^*}$$

componentwise yields a (n, t) -sharing of $x + x^*$, i.e.,

$$(x_1 + x_1^*, \dots, x_n + x_n^*; yy^*) \in D_{x+x^*}$$

Remark B.7. (*Semantic security*) Given $g \in \mathbb{G}$, for any two secrets $x, x^* \in \mathbb{Z}_q$ and index set $J \subset \{1, \dots, n\}$ with $|J| = t - 1$, the probability distributions

$$\{\{x_i\}_{i \in J} : (x_1, \dots, x_n; y) \leftarrow D(g, x)\}, \quad \{\{x_i\}_{i \in J} : (x_1, \dots, x_n; y) \leftarrow D(g, x^*)\}$$

are computationally indistinguishable.

C One-more discrete-logarithm (OMDL) hardness

The OMDL problem family is a parametrized extension of the discrete-logarithm (DL) problem, yielding a sequence of increasingly stronger hardness assumptions. It was introduced in [8] and [6] as a way to capture design properties that pragmatically guarantee security under DL hardness while being themselves irreducible to the latter. Note that reducing security to a stronger assumption amounts to enhanced capabilities for the attacker, i.e., confining ourselves to a narrower class of black-box adversaries. This represents a tradeoff between the higher plausibility of the weaker assumption and having a security proof at hand, which may otherwise be intractable; under the stronger assumption a formal security proof may become possible, while a similar proof under the weaker assumption may not even exist. Since the solvability of the harder problem implies solvability of the easier one, this proof may be regarded as evidence that security holds true under the weaker assumption.

A DL-oracle for the group $\mathbb{G} = \langle g \rangle$ is a polynomial-time procedure $z \leftarrow \mathcal{O}_{\mathbb{G}}^{\text{dlog}}(u)$, $u \in \mathbb{G}$, such that $u = g^z$. Given $t \geq 1$, we consider the following problem.

Game C.1. ($\text{OMDL}_{\mathbb{G}, t-1}$ – *One-more discrete-logarithm problem*, $t \geq 1$) Given an adversary $\mathcal{A}$ with challenger $\mathcal{C}$,

- $\mathcal{C}$ samples $w_i \leftarrow_{\$} \mathbb{G}$, $0 \leq i \leq t - 1$, and sends them to $\mathcal{A}$

- $\mathcal{A}$ outputs $z_0, \dots, z_{t-1}$

$\mathcal{A}$ wins if $\bigwedge_{i=0}^{t-1} w_i = g^{z_i}$ and it has issued at most $t - 1$ queries to $\mathcal{O}_{\mathbb{G}}^{\text{dlog}}$.

The probability that $\mathcal{A}$ solves $\text{OMDL}_{\mathbb{G}, t-1}$ is denoted by

$$\text{Adv}_{\mathbb{G}, t-1}^{\text{OMDL}}[\mathcal{A}]. \tag{C.2}$$

$(t - 1)$ -OMDL *hardness in* $\mathbb{G}$ is the assumption that (C.2) is negligible for every polynomial-time adversary. Evidently, 0-OMDL is equivalent to DL. If an adversary solves $\text{OMDL}_{\mathbb{G}, t^*-1}$ for $t^* > t$, then it trivially solves $\text{OMDL}_{\mathbb{G}, t-1}$. In particular, $(t - 1)$ -OMDL, $t > 1$, is generally a stronger assumption than DL.

D Hash oracles

Algorithm 10 $\mathcal{O}^{\text{hash}}(u)$

```

1: if  $u \notin \text{Dom}(\text{Map}_1)$ 
2:    $c \leftarrow h[\nu]$ 
3:    $\text{Map}_1[u] \leftarrow c$ 
4:    $\text{Map}_2[u] \leftarrow \nu$ 
5:    $\nu \leftarrow \nu + 1$ 
6: else
7:    $c \leftarrow \text{Map}_1[u]$ 
8: return  $c$ 

```

When querying a random oracle H , we usually need to keep track of distinct oracle queries. To this end, given a finite tape of preallocated hashes $h = [h_1 \dots, h_m]$ and initially empty lookup tables Map_1 , Map_2 , we define the *hash oracle* $\mathcal{O}^{\text{hash}}$ to be Algorithm 10. It functions like a random oracle with lookup table Map_1 , but draws hashes from h rather than sampling them uniformly; upon inserting a fresh hash to Map_1 , it takes care to update the tape pointer so that the next predefined hash be available on demand and insert the current pointer to Map_2 so that the respective query can be tracked by order. Evidently, if $h_1, \dots, h_n$ are uniformly sampled from the codomain of H , then $\mathcal{O}^{\text{hash}}$ cannot be distinguished from H . We state this remark formally for the sake of reference.

Lemma D.1. (*Random oracle simulation*) *Given a random oracle H with codomain C and initially empty lookup table Map , initialize a hash oracle $\mathcal{O}^{\text{hash}}$ over an empty tape h of size $m \geq 1$ and empty lookup tables Map_1 and Map_2 . For any distinct elements $u_1, \dots, u_m$, the probability distribution*

$$\{(c_1, \dots, c_m) : c_\nu \leftarrow \mathcal{O}^{\text{hash}}(u_\nu), h[\nu] \leftarrow_{\S} C, \nu = 1, \dots, m\}$$

is identical to the distribution $\{(c_1, \dots, c_m) : c_\nu \leftarrow H(u_\nu), \nu = 1, \dots, m\}$.

Proof. Follows directly from the definition of $\mathcal{O}^{\text{hash}}$, Algorithm 10. □

E Local Forking Lemma

The Forking Lemma was introduced in [29] as part of the rewinding methodology for proving the unforgeability of signatures that are based on the Fiat-Shamir transform. It was later elaborated as General Forking Lemma by [7], who abstracted away the signature specific details. The Local Forking Lemma is a variant by [5] that dispenses with rewinding completely, allowing to fork a process by reprogramming the oracle at a single location as opposed to every location after the fork. Consider a deterministic algorithm of the form

$$\mathcal{S} : \Theta \times C^m \rightarrow \{0, 1, \dots, m\} \times T, \quad (k, \tau) \leftarrow \mathcal{S}(\theta; h_1, \dots, h_m), \quad (\text{E.1})$$

running internally a hash oracle with tape $h_1, \dots, h_m$ (cf. Appendix D). Think of θ as the random coins, k as the *main* output and τ as the *side* output. If $k \geq 1$, then the main output

can be thought of as indicating some peculiarity of the k -th distinct oracle query regarding the side output, while the case $k = 0$ is interpreted as failure. We denote by

$$\text{acc}(\mathcal{S}) = \Pr[k \geq 1 : \left\{ \begin{array}{l} (k, \tau) \leftarrow \mathcal{S}(\theta; h_1, \dots, h_m), \quad \theta \leftarrow_{\$} \Theta \\ h_\nu \leftarrow_{\$} C, \quad 1 \leq \nu \leq m \end{array} \right\}] \quad (\text{E.2})$$

the probability that $\mathcal{S}$ accepts taken over the possible coins and oracle outputs.

Definition E.3. The *local fork of $\mathcal{S}$* is Algorithm 11.

Algorithm 11 LocalFork $_{\mathcal{S}}(\theta)$

```

1:  $h_\nu \leftarrow_{\$} C, \quad 1 \leq \nu \leq m$ 
2:  $(k, \tau) \leftarrow \mathcal{S}(\theta; h_1, \dots, h_m)$ 
3: if  $k = 0$ 
4:   return  $(0, \perp)$ 
5:  $h_k^* \leftarrow_{\$} C$ 
6:  $(k^*, \tau^*) \leftarrow \mathcal{S}(\theta; h_1, \dots, h_{k-1}, h_k^*, h_{k+1}, \dots, h_m)$ 
7: if  $k^* = k \wedge h_k \neq h_k^*$ 
8:   return  $(1, (\tau, \tau^*))$ 
9: return  $(0, \perp)$ 

```

That is, if $\mathcal{S}$ accepts, then the algorithm repeats its execution with the same input except for the k -th hash, which is afresh uniformly sampled; otherwise it aborts. When the second execution completes, the algorithm checks whether $\mathcal{S}$ accepts again under the constraint $h_k^* \neq h_k$ and returns both side outputs; otherwise, it aborts. We denote by

$$\text{forkAcc}(\mathcal{S}) = \Pr[b = 1 : (b, \text{out}) \leftarrow \text{LocalFork}_{\mathcal{S}}(\theta), \quad \theta \leftarrow_{\$} \Theta] \quad (\text{E.4})$$

the probability that the algorithm does not abort taken over the random coins of $\mathcal{S}$. The local forking lemma asserts the following lower bound.

Lemma E.5. (*Local Forking Lemma*, [5]) *For every deterministic algorithm $\mathcal{S}$ as in (E.1) issuing $m \geq 1$ distinct queries to a hash oracle with tape $[h_1, \dots, h_m]$,*

$$\text{forkAcc}(\mathcal{S}) \geq \frac{1}{m} \text{acc}(\mathcal{S})^2 \quad (\text{E.6})$$

References

- [1] Mehmet Adalier. Efficient and Secure Elliptic Curve Cryptography Implementation of Curve P-256. In *NIST Workshop on Cybersecurity in a Post-Quantum World*, 2015. Presented at the NIST Workshop on Cybersecurity in a Post-Quantum World.
- [2] Renas Bacho and Alireza Kavousi. Sok: Dlog-based distributed key generation. *2025 IEEE Symposium on Security and Privacy (SP)*, pages 614–632, 2025.

- [3] Karim Bagheri, Noah Knapen, Georgio Nicolas, and Mahdi Rahimi. Pre-constructed publicly verifiable secret sharing and applications. In *Applied Cryptography and Network Security (ACNS)*, 2025. To appear.
- [4] K. Barr and K. Asanovic. Energy aware lossless data compression. In *MobiSys*. ACM, 2003.
- [5] Mihir Bellare, Wei Dai, and Lucy Li. The Local Forking Lemma and its Application to Deterministic Encryption. Cryptology ePrint Archive, Paper 2019/1017, 2019.
- [6] Mihir Bellare, Chanathip Namprempre, David Pointcheval, and Michael Semanko. The One-More-RSA-Inversion Problems and the Security of Chaum’s Blind Signature Scheme. *J. Cryptology*, 16:185–215, 2003.
- [7] Mihir Bellare and Gregory Neven. Multi-signatures in the plain public-Key model and a general forking lemma. In *Conference on Computer and Communications Security*, 2006.
- [8] Mihir Bellare and Adriana Palacio. GQ and Schnorr Identification Schemes: Proofs of Security against Impersonation under Active and Concurrent Attacks. In *Advances in Cryptology - CRYPTO 2002, 22nd Annual International Cryptology Conference, Santa Barbara, California, USA, August 18-22, 2002, Proceedings*, volume 2442 of *Lecture Notes in Computer Science*, pages 162–177. Springer, 2002.
- [9] Daniel J. Bernstein. Pippenger’s Exponentiation Algorithm. *Unpublished manuscript*, 2002. Survey paper on Pippenger’s exponentiation algorithms.
- [10] Dan Boneh, Manu Drijvers, and Gregory Neven. Compact multi-signatures for smaller blockchains. In *Advances in Cryptology – CRYPTO 2018*, volume 10992 of *Lecture Notes in Computer Science*, pages 435–464. Springer, 2018.
- [11] Sourav Das, Zhuolun Xiang, Lefteris Kokoris-Kogias, and Ling Ren. Practical asynchronous high-threshold distributed key generation and distributed polynomial sampling. In *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, 2023.
- [12] Sourav Das, Thomas Yurek, Zhuolun Xiang, Andrew Miller, Lefteris Kokoris-Kogias, and Ling Ren. Practical asynchronous distributed key generation. In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 2518–2534. IEEE, 2022.
- [13] Paul Feldman. A practical scheme for non-interactive verifiable secret sharing. *28th Annual Symposium on Foundations of Computer Science (sfcs 1987)*, pages 427–438, 1987.
- [14] Rosario Gennaro, Stanislaw Jarecki, Hugo Krawczyk, and Tal Rabin. Secure distributed key generation for discrete-log based cryptosystems. *Journal of Cryptology*, 20:51–83, 1999.
- [15] Conrado Gouvêa. Frost performance. <https://zfnd.org/frost-performance>, 2023. Accessed: 2025-09-11.
- [16] Jens Groth. On the size of pairing-based non-interactive arguments. In *Advances in Cryptology - EUROCRYPT 2016*, volume 9666 of *Lecture Notes in Computer Science*, pages 305–326. Springer, 2016.
- [17] Kobi Gurkan, Philipp Jovanovic, Mary Maller, Sarah Meiklejohn, Gilad Stern, and Alin Tomescu. Aggregatable distributed key generation. *Cryptology ePrint Archive, Paper 2021/005*, 2021.
- [18] Darrel Hankerson, Alfred J. Menezes, and Scott Vanstone. *Guide to Elliptic Curve Cryptography*. Springer Professional Computing. Springer-Verlag, New York, 2004.

- [19] Aniket Kate, Yizhou Huang, and Ian Goldberg. Distributed key generation in the wild. Cryptology ePrint Archive, Paper 2012/377, 2012.
- [20] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In *Advances in Cryptology - ASIACRYPT 2010*, volume 6477 of *Lecture Notes in Computer Science*, pages 177–194. Springer, 2010.
- [21] Marcel Keller, Gert Læssøe Mikkelsen, and Andy Rupp. Efficient Threshold Zero-Knowledge with Applications to User-Centric Protocols. Cryptology ePrint Archive, Paper 2012/306, 2012.
- [22] Chelsea Komlo and Ian Goldberg. FROST: Flexible Round-Optimized Schnorr Threshold Signatures. Cryptology ePrint Archive, Paper 2020/852, 2020.
- [23] Ueli M. Maurer. Unifying Zero-Knowledge Proofs of Knowledge. In Bart Preneel, editor, *Progress in Cryptology - AFRICACRYPT 2009, Second International Conference on Cryptology in Africa, Gammarrth, Tunisia, June 21-25, 2009. Proceedings*, volume 5580 of *Lecture Notes in Computer Science*, pages 272–286. Springer, 2009.
- [24] Bodo Möller. Algorithms for Multi-exponentiation. In Serge Vaudenay and Amr M. Youssef, editors, *Selected Areas in Cryptography*, volume 2259 of *Lecture Notes in Computer Science*, pages 165–180, Berlin, Heidelberg, 2001. Springer.
- [25] Computer Security Resource Center NIST. Multi-party threshold cryptography. <https://csrc.nist.gov/projects/threshold-cryptography>, 2025. Accessed: 2025-09-11.
- [26] Dong-won Park, Nam Su Chang, Sangyub Lee, and Seokhie Hong. Fast Implementation of NIST P-256 Elliptic Curve Cryptography on 8-Bit AVR Processor. *Applied Sciences*, 10(24):8816, 2020. DOI: 10.3390/app10248816.
- [27] Torben P. Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In *Annual International Cryptology Conference*, 1991.
- [28] Nicholas Pippenger. On the Evaluation of Powers and Monomials. *SIAM Journal on Computing*, 9(2):230–250, May 1980.
- [29] David Pointcheval and Jacques Stern. Security Proofs for Signature Schemes. In *Advances in Cryptology - EUROCRYPT '96, International Conference on the Theory and Application of Cryptographic Techniques, Saragossa, Spain, May 12-16, 1996, Proceeding*, volume 1070 of *Lecture Notes in Computer Science*, pages 387–398. Springer, 1996.
- [30] Claus-Peter Schnorr. Efficient signature generation by smart cards. *Journal of Cryptology*, 4:161–174, 2004.
- [31] Berry Schoenmakers. A simple publicly verifiable secret sharing scheme and its application to electronic voting. In *Annual International Cryptology Conference*, 1999.
- [32] Adi Shamir. How to Share a Secret. *Commun. ACM*, 22(11):612–613, 1979.
- [33] Markus Stadler. Publicly verifiable secret sharing. In Ueli Maurer, editor, *Advances in Cryptology — EUROCRYPT '96*, volume 1070 of *Lecture Notes in Computer Science*, pages 190–199. Springer-Verlag, 5 1996.
- [34] Douglas R. Stinson and Reto Stroh. Provably Secure Distributed Schnorr Signatures and a (t, n) Threshold Scheme for Implicit Certificates. In Vijay Varadharajan and Yi Mu, editors, *Information Security and Privacy*, pages 417–434, Berlin, Heidelberg, 2001. Springer Berlin Heidelberg.
- [35] Alin Tomescu, Robert Chen, Yiming Zheng, Ittai Abraham, Benny Pinkas, Guy Golan Gueta, and Srinivas Devadas. Towards scalable threshold cryptosystems. In *2020 IEEE Symposium on Security and Privacy (SP)*, pages –. IEEE, 2020.